



Architecture SOA et Services Web

Auditoire :

3ème année Licence Génie Logiciel et Système d'information

Enseignante :

Dr. Ghada Feki

Maitre Assistante à la Faculté des Sciences de Monastir

Email : ghada.feki@gmail.com

Plan

- Chapitre 1 : Introduction
- Chapitre 2 : Les Services
- Chapitre 3 : Web Services SOAP
- Chapitre 4 : Web Services REST
- Chapitre 5 : Orchestration & Chorégraphie

Chapitre 1 : Introduction

Problématique

Un système d'information d'entreprise est un assemblage d'applications indépendantes.

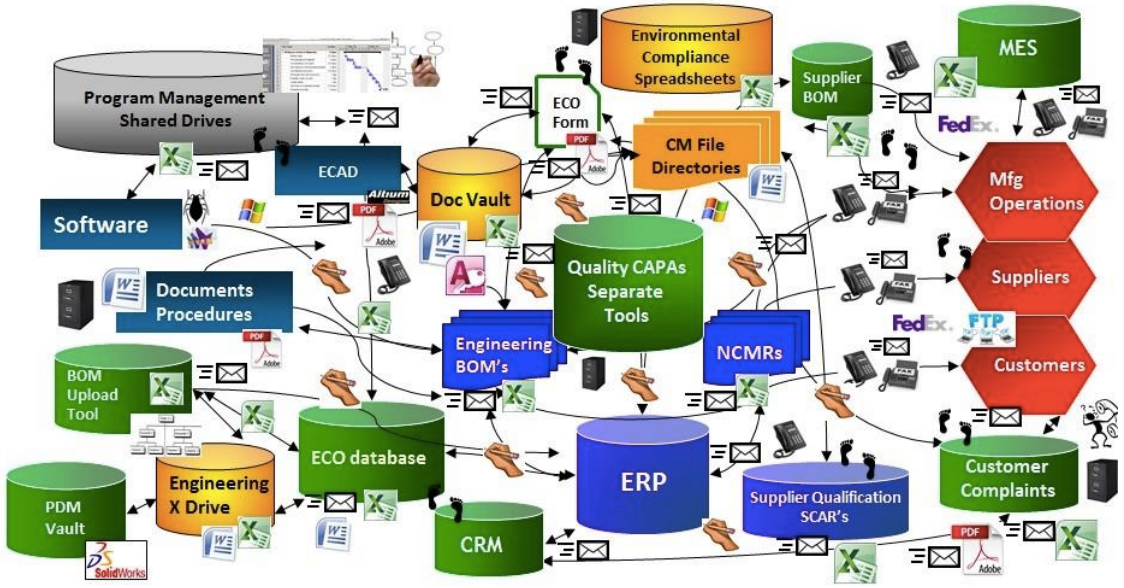
Besoin de communication, d’échange entre ces applications et d’intégration de nouvelles applications généralement hétérogènes.



Problématique

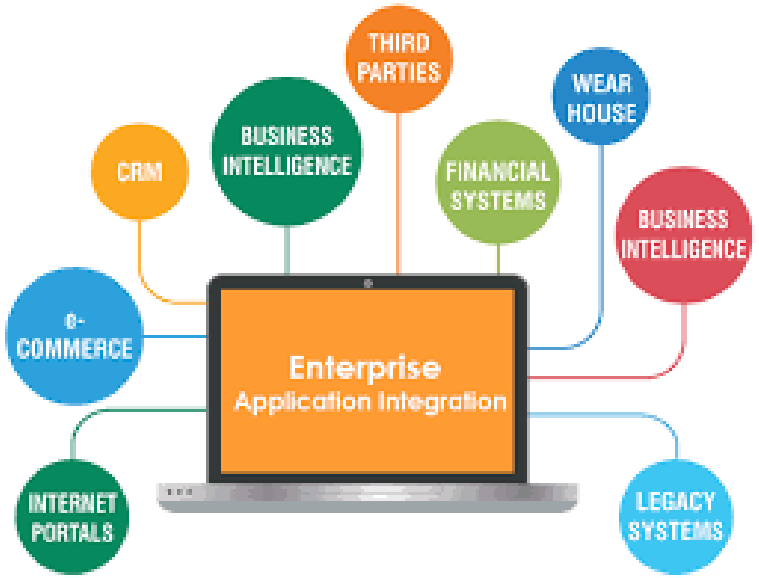
Parmi les solutions existantes pour faire communiquer ces applications indépendantes, et intégrer de nouvelles applications, on trouve:

- L'approche spaghetti: une communication point à point



Problématique

- L’EAI (Enterprise Application Integration) : Réunir les différents programmes d'une organisation au sein d'un même espace. Elle vient remplacer l’approche spaghetti et alléger l’architecture d’interconnexion en mettant à la disposition des applications un nœud centrale de communication.



Problématique

Limites de ces solutions :

- Développement coûteux
- Interconnexions redondantes
- Grande complexité
- Réutilisation et maintenance difficile

Solution

Naissance de la notion des **services Web** qui:

- Garantit L'interopérabilité
- Fait interagir des composants hétérogènes, distants, et indépendants
- Simplifie la communication entre ces composants
- Se base sur des technologies qui existent déjà (XML, HTTP)

Définitions

- Service: « Un service est un comportement défini par contrat, qui peut être réalisé et fourni par tout composant pour être utilisé par tout composant sur la base unique du contrat. »



Définitions

- Service Web : entité logicielle mise à disposition sur l'internet ou sur un réseau privé, publiable et accessible en utilisant le langage XML et les protocoles standards du web. Elle est indépendante du système d'exploitation et du langage de programmation.

- Architecture SOA (Service-Oriented Architecture) : modèle de conception qui rend des composants logiciels réutilisables, grâce à des interfaces de services qui utilisent un langage commun pour communiquer via un réseau.

Avantages

- Mise sur le marché accélérée et plus grande flexibilité : le caractère réutilisable des services facilite et accélère le regroupement des applications. Les développeurs ne doivent plus repartir de zéro à chaque fois, comme c'est le cas pour les applications monolithiques.
- Utilisation de l'infrastructure existante sur les nouveaux marchés : grâce à l'architecture SOA, les développeurs peuvent plus facilement étendre et mettre à l'échelle les fonctionnalités d'une plateforme ou d'un environnement.
- Coûts réduits grâce à une meilleure agilité et à un développement plus efficace.

Avantages

- Maintenance facilitée : les services étant autonomes et indépendants, ils peuvent être modifiés et mis à jour autant que nécessaire, sans affecter les autres services.
- Évolutivité : comme l'architecture SOA s'adapte à plusieurs services, plateformes et langages de programmation, l'évolutivité est considérablement accrue.
- Fiabilité améliorée : il est plus facile de déboguer des petits services plutôt qu'un long code, ce qui permet de créer des applications plus fiables.
- Grande disponibilité : les fonctions de l'architecture SOA sont à la portée de tous.

Inconvénients

- Assez coûteuse en termes de main-d'œuvre : Plusieurs équipes d'unités informatiques et de développeurs sont nécessaires pour configurer et gérer chaque code de service. Bien que ce coût soit principalement supporté par les prestataires de services tiers, il peut en fin de compte se traduire par des coûts financiers pour l'entreprise et ses clients.

Inconvénients

- Surcharge du trafic de données : Dans le cadre des protocoles de sécurité, les commandes et les entrées doivent être vérifiées avant d'être transférées au tiers. À mesure que le nombre de services sur une architecture SOA augmente, le nombre d'entrées et de codes de vérification pour ces entrées augmente également. La quantité supplémentaire de trafic de données chargera votre système.
- Besoin en ressources plus élevé : L'élément d'une charge de données plus élevée nécessite une bande passante plus importante pour garantir que la vitesse du réseau est suffisante pour assurer la satisfaction du client.

Top Business utilisant SOA

- Hôtels qui externalisent les services de blanchisserie et de restauration
- Entreprises géantes utilisant des fournisseurs de services tiers
- Tout site Web utilisant PayPal ou d'autres services de paiement
- Site de vente utilisant un service de livraison en ligne
- Un magasin de médias / livres utilisant iTunes, Amazon ou d'autres systèmes de vente
- Le système de réservation de la SNCF (recherche d'horaire, demande de tarif, réservation…) prend en charge à la fois les terminaux des guichets des agences et gares, et les sollicitations de son site web de commande en ligne

Exemple





A good example would an online food delivery service.
The food delivery company can use a Paypal for payment service, FedEx for delivery, and Bluehost for Webhosting.



Concepts d’une architecture SOA

- Un service web est une composante web qui possède une interface lui permettant la communication.

- Une interface est un ensemble de fonctions qu’on peut les invoquer suivant une spécification bien définie.

- Un service est conçu/développé par un fournisseur et consommé par un consommateur.

Acteurs d’une architecture SOA

- **Le fournisseur de service** (Service Provider) : Application s'exécutant sur un serveur et comportant un module logiciel accessible par Internet en XML.

- **Le client** (Service Requester) : Application cliente se liant à un service et invoquant ses fonctions par des messages XML (SOAP).

- **L’annuaire de services** (Service Registery) : Annuaire des services publiés par les providers (UDDI).

Acteurs d’une architecture SOA

- Le **consommateur** a accès à l’interface du service pour invoquer une fonctionnalité particulière.
- Le **fournisseur** a accès au code source du service dans le but de faire une mise à jour ou une maintenance du service.
- Le fournisseur publie son service dans un registre de services.
- Le consommateur cherche la disponibilité d’un service particulier dans ce registre.
- Pour invoquer un service, le consommateur doit respecter la spécification du service (ou aussi dit son contrat).

Fonctionnement

Le fournisseur de service (Service Provider) est chargé de :

1) Définir le service: implémentation, description, contrat d'utilisation.

2) Publier sa description dans l’annuaire

3) Réaliser les opérations

Fonctionnement

L’annuaire (Entrepôt) (Discovery Agency) est chargé de :

1) Recevoir et enregistrer les descriptions de services publiées par les fournisseurs

2) Recevoir et répondre aux recherches de services lancées par les clients

Fonctionnement

Le client (Service Requestor) est chargé de :

1) Lancer la recherche sur un service approprié.

2) Obtenir la description du service grâce à l'annuaire.

3) Utiliser le service

SOA vs Programmation distribuée et POO

Caractéristiques de la programmation orientée objets

- La programmation orientée objets est basée sur le principe d'encapsulation d'un ensemble d'informations dans un objet doté d'un ensemble de méthodes permettant la manipulation de ses informations.

- Ce style de programmation à un grand avantage surtout du côté séparation du code ce qui facilite largement les taches suivantes du développement du logiciel.

SOA vs Programmation distribuée et POO

Caractéristiques de la programmation orientée objets

Limites de cette architecture :

1. Structure et architecture de l'application peu visible.
2. Interactions entre objets enfouies dans le code.
3. Évolution / modification difficile.
4. Gestion de la consistance d'un changement délicate
5. Granularité encore trop fine: ce qui la rend mal adaptée à la programmation à grande échelle.
6. Couplage fort: ce qui rend difficile la réutilisation et accroît la complexité des Systèmes Orientés Objet.

SOA vs Programmation distribuée et POO

Caractéristiques des applications distribuées (orientées composants)

Une application distribuée est formée d'un ensemble de composants logiciels qui :

1) Collaborent pour l'exécution de tâches communes
2) Distants géographiquement
3) Interconnectés via un réseau de communication
4) Hétérogènes
5) Solutions qui ont fait leur preuve: DCOM, CORBA, EJB, RMI, .Net, …

SOA vs Programmation distribuée et POO

Caractéristiques des applications distribuées (orientées composants)

Les faiblesses de ces solutions peuvent être résumées dans :

1) Format de représentation de données spécifiques

2) Interopérabilité uniquement si les composants utilisent la même solution

3) Parfois elle utilise des protocoles de transport spécifiques ce qui nécessite une configuration spécifique du réseau.

4) Couplage fort : phénomène du plat de spaghetti

SOA vs Programmation distribuée et POO

Architecture orientée service

1) Elle partage l'aspect modulaire (ensemble de fonctionnalités qui font sens) avec la programmation orientée objets.

2) Elle partage les caractéristiques suivantes d'un composant (application distribuée) :

- Boite noire (séparation interface/implémentation)
- Indépendant de la localisation
- Neutralité vis-à-vis des protocoles de transport

SOA vs Programmation distribuée et POO

Architecture orientée service

3) Correspond à un périmètre fonctionnel que l'on souhaite exposer à des consommateurs.

4) Services faiblement couplés (indépendant des autres services).

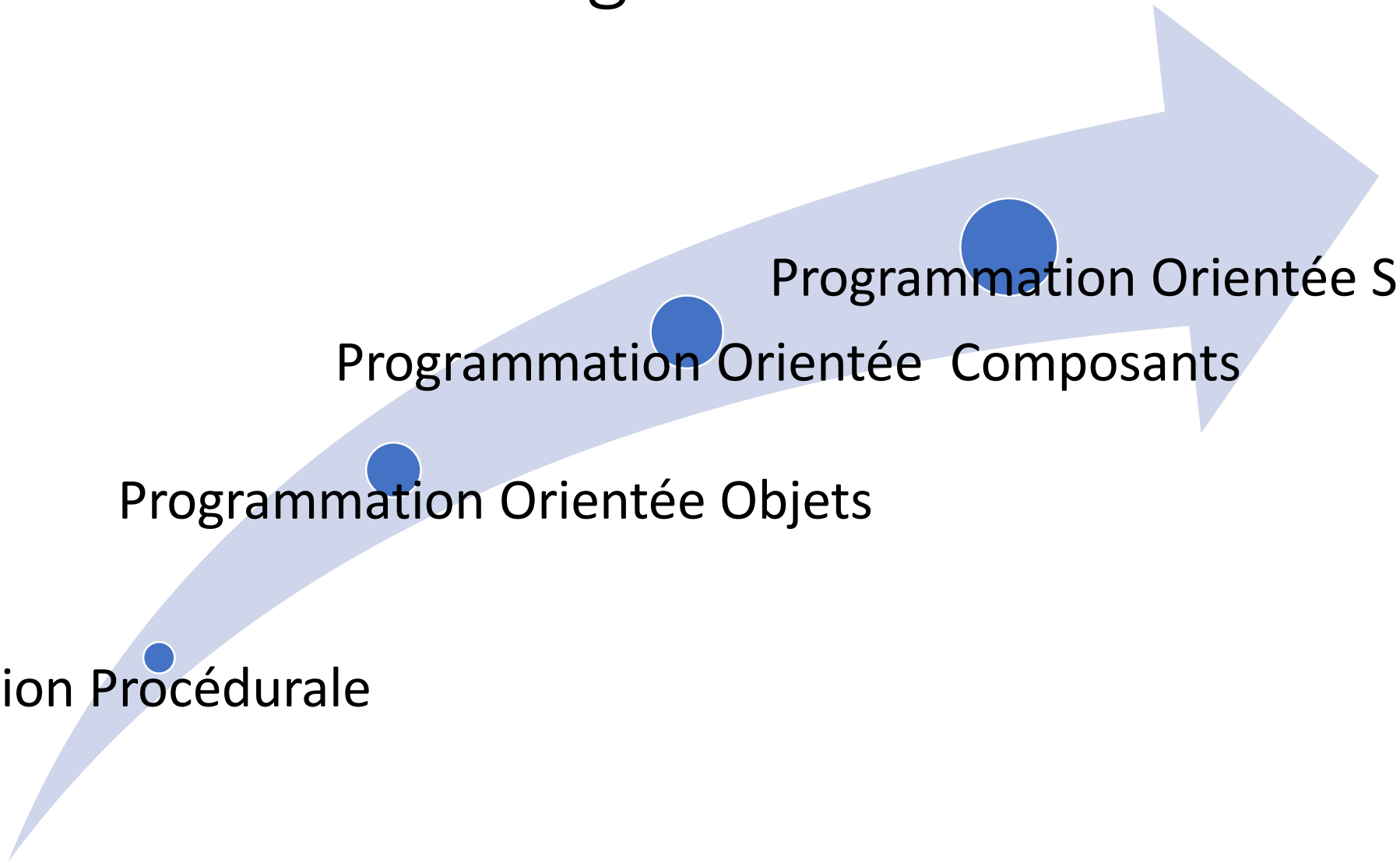
5) Expose un petit nombre d'opérations offrant un traitement de bout en bout.

6) Utilisation de standards.

7) Pas de remise en cause de l'existant lors d'évolutions technologiques

8) Découplage entre fournisseur et consommateur de services.

Niveau d’abstraction grandissant

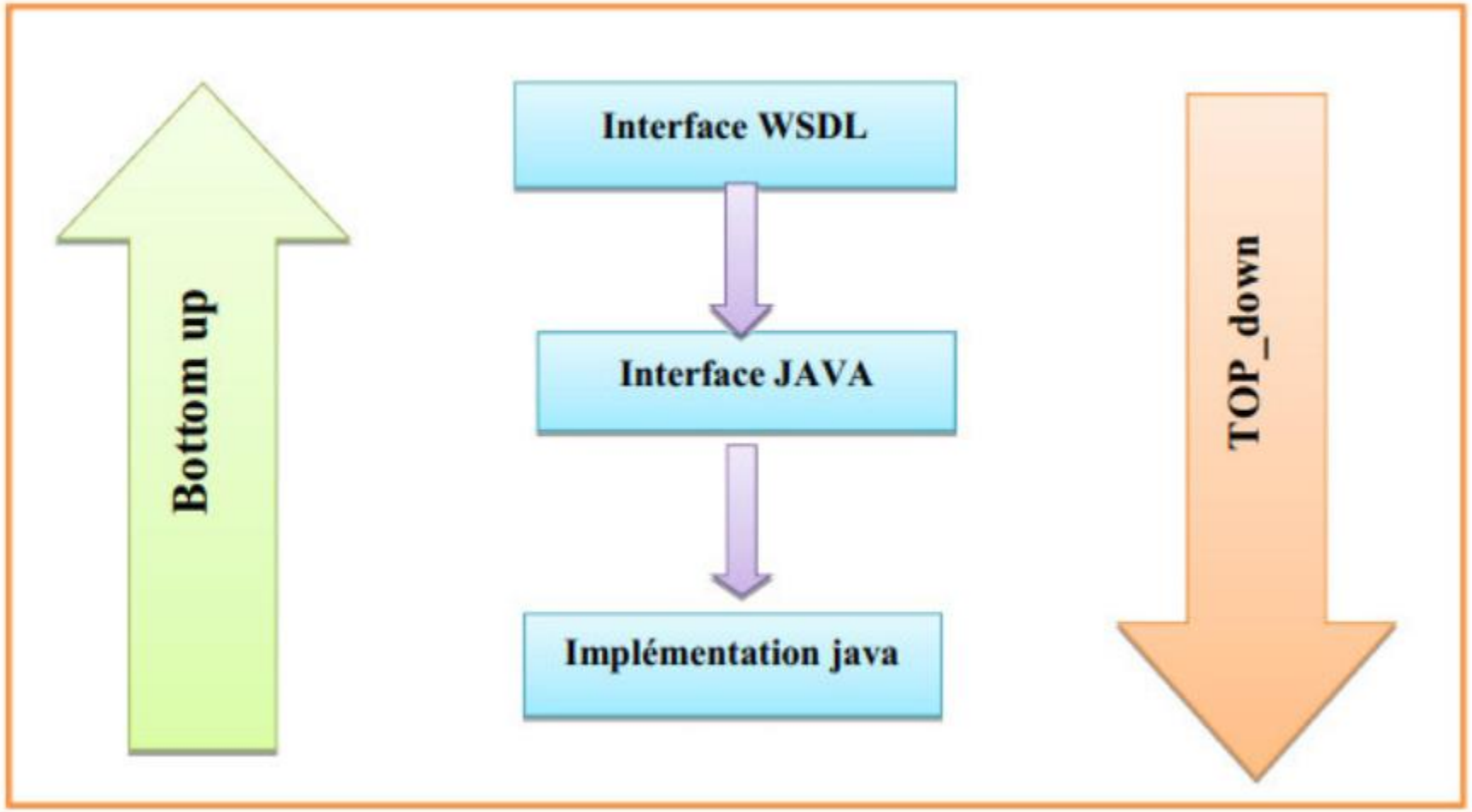


Approches de SOA

Deux approches sont définies pour implémenter l'architecture SOA :

- **Bottom-Up :** Son principe est de commencer par un petit groupe et ajouter jusqu'à on obtient une grande entreprise.
- **Top-down :** c'est l'adverse de Bottom-Up. Top-down est la décomposition d'un système ou problème jusqu'à l'obtient d'un petit service de base.

Approches de SOA



Bilan SOA - Avantages

1. Accès à distance via un navigateur Web.
2. Pas de logiciel sur le poste client.
3. Possibilité d'interagir par n'importe quel appareil connecté à Internet.
4. Compatibilité avec différents systèmes d'exploitation et différentes plates-formes (utilisation des standards).
5. Système d'information flexible (anticiper les évolutions technologiques). Il suffit de faire évoluer un service ou d'ajouter un nouveau service sans avoir de perturbations pour les autres services.
6. Réutilisation de code.
7. Une plus grande tolérance aux pannes (maintenance facile).

Bilan SOA - Inconvénients

1. Recentralisation des systèmes (charge très importante sur le serveur).
2. Coûts de conception et de développement initiaux importants à la fois financier et humain (formation d'une équipe d'experts pour la conception et des équipes pour l'implémentation et l'administration).
3. Lenteur d'exécution (services asynchrones).
4. Manque de maturité de standards (course à la spécification entre W3C et OASIS).

Chapitre 2 : Les Services

Définitions

- Un service est un comportement défini par contrat, qui peut être réalisé et fourni par tout composant pour être utilisé par tout composant sur la base unique du contrat.
- Un Service est un composant logiciel distribué, exposant les fonctionnalités à forte valeur ajoutée d'un domaine métier.
- Les aspects d'un service sont : le contrat standardisé, le couplage lâche, l'abstraction, la réutilisabilité, l'autonomie, sans état, découvrabilité, et composabilité.

Les aspects d’un service

Un service est un composant logiciel distribué satisfaisant les huit aspects (caractéristiques) suivants :

1) Il est exposé à travers un contrat standardisé.
2) Il permet un couplage lâche entre le consommateur et le fournisseur.
3) Il est abstrait (vu comme une boîte noire).
4) Il est réutilisable.
5) Il est autonome.
6) Il est sans état.
7) Il est découvrable (Possibilité de découverte de service).
8) Il est composable (Capacité de composition en modules de services).

Les aspects d’un service
Contrat Standardisé

L’ensemble des services d’un même Système Technique sont exposés au travers de **contrats** respectant les mêmes règles de **standardisation**.

Qu’est-ce qu’un contrat de service ?

Les aspects d'un service
Contrat Standardisé

Le contrat de service définit un accord entre le fournisseur et le consommateur. Il est composé :

- D'un **contrat syntaxique** qui propose une représentation technique du service : Il constitue le contrat d'utilisation du service (son interface). Il présente le nom du traitement, ses paramètres d'entrée et de sortie et les contraintes structurelles (format et contrainte sur les données) qui s'y appliquent.

Les aspects d'un service
Contrat Standardisé

- D'un **contrat sémantique** qui fournit une description informelle du traitement : Il précise les règles et contraintes d'utilisation du service : La valorisation des messages de réponse; Les exceptions; les pré-conditions et post-conditions techniques ou métiers.
- D'un **contrat de niveau de service (QoS et SLA)** qui précise les engagements du service : Il spécifie par exemple le temps de réponse maximum attendu, les plages horaires d'accessibilité, le temps de reprise après interruption, les procédures mises en œuvre en cas de panne, les procédures de prise en charge du support, …

QoS : Quality of service

SLA : Service Level Agreement

Les aspects d'un service
Contrat Standardisé

Exemple :

Pour un service web (service implémenté sous forme de web service), son contrat peut être formé comme suit :

- Un document WSDL décrivant l'interface du service (les modalités d'accès), ceci fait partie du contrat syntaxique.
- Un ensemble de « policies » qui définissent les règles d'utilisation du service :
    - Les policies font parties du contrat sémantique. Les règles et contraintes qu'elles expriment adressent des domaines variés : Sécurité, encodage, langue, versioning, métiers, …

Les aspects d'un service
Contrat Standardisé

- Un ensemble de schémas XML (XSD) qui définissent les types de données échangées par le service :
  - Les XSD font partie du contrat syntaxique. Elles définissent les formats de données et les contraintes structurelles.
  - Les XSD font également partie du contrat sémantique. En effet, la richesse des types et restrictions XSD permet d'auto-documenter les valeurs possibles et permet souvent d'éviter des confusions.

XML : Extensible Markup Language

XSD : XML Schema Definition

Les aspects d’un service
Contrat Standardisé

- Documentations complémentaires.
  - Ces documentations complètent le contrat sémantique. C’est par exemple dans les spécifications que l’on décrira les prés et post condition d’utilisation du service.
  - Ces documentations constituent le contrat de niveau de service (QoS & SLA). Notons que l’on parle bien d’un contrat qui définit un ensemble d’indicateurs et de valeurs seuils.

Les aspects d'un service
Contrat Standardisé

Standardisation du contrat :

Le contrat de service est donc une notion complexe qui ne se limite pas à la (simple) définition d'interfaces. Un contrat de service est constitué de nombreux éléments (techniques ou non) qui forment un fond documentaire pour lequel il est préférable (voire indispensable) de respecter un formalisme commun. L'utilisation de ce formalisme commun est le meilleur moyen de construire un modèle cohérent et donc facile à comprendre et à partager.

Les aspects d'un service
Contrat Standardisé

En plus de cadrer la définition des contrats (et donc de faciliter la communication), l'utilisation d'un formalisme commun et de règles de standardisation facilite la centralisation des éléments constituant les contrats. Cette centralisation encourage la réutilisation.

Parmi les constituants d'un contrat de service, les règles d'utilisation (policies) et les formats des données (XSD) sont des bons candidats à la réutilisation vu qu'on général les mêmes règles d'utilisations et les mêmes structures de données sont partagées entre les services dans un seul système.

Il est indispensable ainsi de séparer les différents éléments du contrat de service afin de permettre leur réutilisation.

Les aspects d’un service
Couplage lâche

- L’objectif de la SOA est de permettre aux entreprises de s’adapter en permanence et être de plus en plus réactives aux variations des marchés et aux avancements technologiques.
- Il est primordial d’éviter les chausse-trappes des grands projets d’architectures distribuées ou intégrées, et en particulier le couplage technique et fonctionnel entre consommateurs et fournisseurs de services.

Les aspects d'un service
Couplage lâche

- **Le couplage technique :** impose au consommateur de connaître le protocole d'échange du fournisseur. À grande échelle, un tel couplage complique, voire interdit l'évolution du socle technique, et risque de figer le SI dans une inertie sclérosante.
- **Le couplage fonctionnel :** impose au client de connaître le format d'échange du fournisseur. Ainsi toute évolution du fournisseur a un impact potentiel sur chacun de ses consommateurs. Mal gérée, cette dépendance peut conduire à de véritables verrous fonctionnels dans le Système d'Informations – interdisant toute évolution ou, plus sûrement, augmentant de façon exponentielle le coût de ces évolutions.

Les aspects d’un service
Couplage lâche

La question qui se pose :

Comment garantir l’indépendance entre le consommateur (Interface client) et le Fournisseur (Service) ?

Sachant que :

La logique d’un service est fortement liée à son environnement d’implémentation

Examinons tout d’abord la nature de **dépendances** qui existent :

Les aspects d'un service
Couplage lâche

- La logique d'un service est directement dépendante de son implémentation (1). Cette implémentation s'appuie sur un ensemble de ressources qui constituent l'environnement d'implémentation du service. La logique d'un service est donc couplée à ces ressources.
- De la même façon, la logique d'un service est fortement liée aux technologies sur lesquelles son implémentation s'appuie (2).
- D'autre part, dans le cadre de services composites, la logique d'un service est également dépendante des services qui le composent (3).
- Enfin, le contrat de service ainsi que sa logique peuvent être couplés aux processus qui les utilisent (4).

Les aspects d'un service
Couplage lâche

Ces couplages sont inévitables.

Du côté consommateur, celui-ci est lié directement avec le service à travers son contrat.

C'est le quatrième type de couplage (Couplage Service/Contrat) qu'il faut réduire afin de permettre un découplage entre le consommateur et le service.

Le contrat d'un service et sa logique peuvent être couplés de deux façons différentes.

1. **Couplage du contrat de service vers la logique d'implémentation du service.**
2. **Couplage de la logique du service au contrat.**

Les aspects d'un service
Couplage lâche

- **Couplage du contrat de service vers la logique d'implémentation du service :**

Un tel couplage résulte d'une approche ***Contract last*** lors du design du service *(le contrat dérive de l'implémentation)*. Cette approche est à exclure car elle induit un couplage du contrat avec l'environnement du service.

- **Couplage de la logique du service au contrat :**

Ce couplage dénote une approche ***Contract first*** dans la conception du service *(le contrat est écrit préalablement à l'implémentation du service)*. Cette approche est toujours préférable et le couplage du service vers son contrat est considéré comme un couplage positif.

Les aspects d'un service
Couplage lâche

Ainsi il faut établir la dépendance dans le bon sens :

- Le consommateur d'un service est lié au contrat de ce dernier, et ne doit être lié qu'à celui-ci *(Pas au service lui-même)*. C'est ce que l'on appelle le **couplage lâche**.
- Or, si le contrat est couplé avec la logique du service, le consommateur va hériter par transitivité de l'ensemble des dépendances de la logique du service avec son environnement. Nous nous retrouvons donc avec **un consommateur qui est fortement couplé au service (*Couplage Fort*)**.

Les aspects d'un service
Abstraction

- Le principe d'abstraction consiste à fournir les services du SI sur un modèle **boîte noire**, or les seules informations fournies au consommateur sont celles contenues dans son contrat, ainsi il est quasiment indispensable de n'y mettre que les informations essentielles à son invocation.
- Il est souvent inutile de fournir les informations complètes sur les détails d'implémentation du service, le fournisseur du service peut permettre un accès contrôlé à ce genre de détails à des concepteurs particuliers de consommateurs.
- Cependant cette restriction d'accès aux informations du service ne doit pas violer le principe de la prédictibilité du service.

Les aspects d'un service
Abstraction

Prédictibilité du service :

- La prédictibilité d'un service signifie que son comportement et la réponse qu'il donne suite à une requête ne doivent pas varier (suite à une concurrence d'accès par exemple).
- Ce principe est induit par **le respect du contrat du service**.
- Ainsi le principe de l'abstraction de service signifie qu'il faut mettre dans le contrat du service toutes et uniquement les informations nécessaires et suffisantes à son invocation.

Les aspects d'un service
Réutilisabilité

- La réutilisabilité des services (et plus largement des ressources du SI) constitue une des pierres angulaires de la mise en œuvre d'une architecture orientée service. En effet, la mise en œuvre d'une architecture SOA vise, entre autres, à éviter le gaspillage des ressources en éliminant les redondances. D'autre part, la réutilisation est une condition première de l'agilisation du SI indispensable à la réduction du time-to-market.

Les aspects d'un service
Réutilisabilité

- La réutilisabilité d'un service désigne la capacité du service à répondre aux besoins de plusieurs types de consommateurs (est-ce que plusieurs consommateurs vont utiliser ce service ?).
- Les services identifiés comme réutilisables sont ainsi mis à la disposition d'un large spectre de services composites, de processus, d'applications, … Au fil du temps, de tels services ont donc vocation à prendre part à un nombre croissant de compositions, d'orchestrations, de workflow, …

Les aspects d'un service
Réutilisabilité

En effet les principaux freins qui peuvent influencer mal sur le degré de réutilisation de service sont :

1. **La centralisation et la standardisation des ressources :**

La **centralisation des contrats de service** garantit qu'un consommateur n'a accès au service que par son contrat, et la **centralisation de la logique métier** permet d'obtenir un inventaire de services hautement normalisé. **La standardisation de cet inventaire** est essentielle pour maximiser la réutilisabilité des services disponibles au sein de l'entreprise.

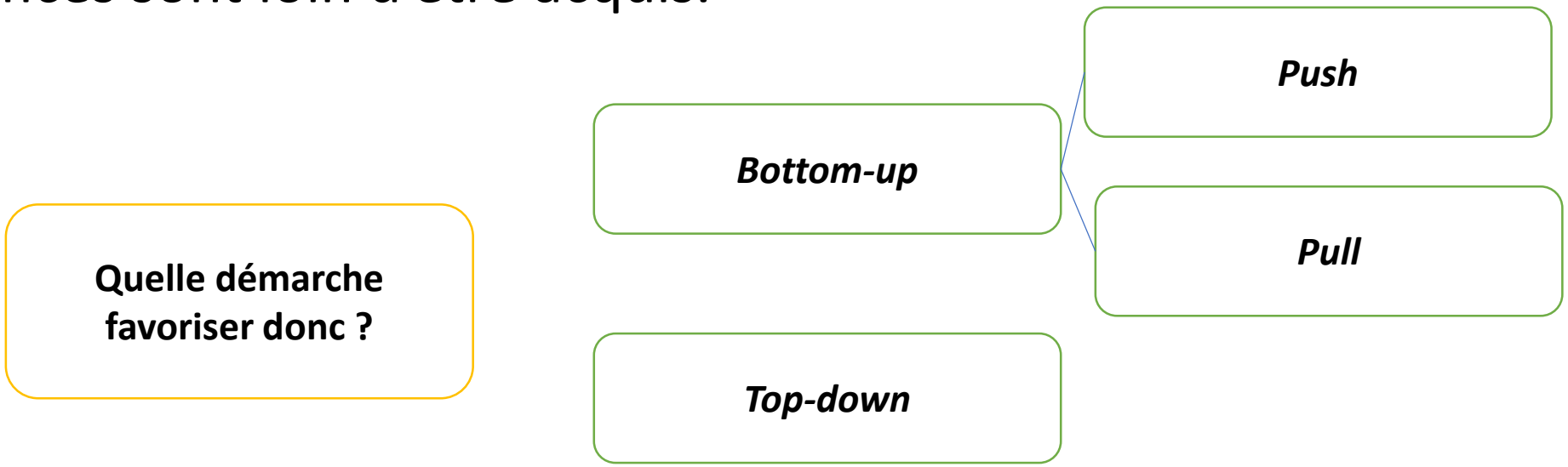
Les aspects d'un service
Réutilisabilité

2. La démarche suivie dans la conception de service :

- **Le processus de conception « *Bottom-up* » :** un service émerge d'une démarche « **Push** » initiée par un fournisseur ou « **Pull** » initiée par un consommateur. Selon l'initiateur de cette démarche (fournisseur ou client), le résultat de ce processus est un service plus générique (**Push**) qui essaye de répondre aux besoins de tous les partenaires (risque d'être sans valeur ajoutée métier), ou bien un service qui répond au besoin d'un client particulier sans forcément prendre en compte des perspectives de réutilisation du service. Cela s'explique par le fait que ce genre de demande se conjugue généralement avec des impératifs « Time To market » imposés par le client du service.

Les aspects d'un service
Réutilisabilité

- **Un processus de conception « *Top-down* » :** le service est le résultat d'une étude plus globale et systémique du SI ou d'un bloc du SI. Cette démarche garantit une **cohérence** globale de l'ensemble des services, néanmoins, sa mise en place s'avère un challenge difficile dont les bénéfices sont loin d'être acquis.



Les aspects d’un service
Réutilisabilité

- Il n’y a pas de démarche à favoriser en particulier.
- Les 3 peuvent coexister selon le contexte et l’offre de services existants. Néanmoins, en phase de conception et de mise en œuvre du service, il est primordial de prendre en considération les points suivants:
  - Favoriser une démarche itérative et collaborative entre clients et fournisseurs de services.
  - Penser un service donné en tant qu’**offre de services**, plutôt qu’un mono-service qui essaie de répondre à tous les besoins.
  - Ne pas considérer l’étude d’urbanisation, s’il y en avait une, comme une cible en elle-même, mais plutôt comme un fil conducteur pour la cible.

Les aspects d'un service
Réutilisabilité

3. La granularité de service :

- La granularité d'un service est définie par le nombre de fonctions recouvertes par celui-ci. En effet ce facteur est très critique, et une mauvaise granularité du service va surement limiter son réutilisation.
- On parle de mauvaise granularité de service lorsque celui-ci couvre trop ou trop peu de fonctionnalités :
  - Un service qui fait trop de choses causera des mauvaises performances et risquerait d'être moins réutilisable à cause des fonctionnalités non utiles pour le consommateur.
  - Un service qui fait trop peu de choses sera sans valeurs ajoutée métier et risque d'être non réutilisable de son besoin à collaborer avec d'autres services pour répondre aux besoins du consommateur.

Les aspects d'un service
Autonomie

- L'autonomie de service signifie que le service n'a pas besoin à d'autres services pour accomplir sa tâche (il est indépendant des autres), et par conséquent ça implique que le comportement d'un service ne dépend pas du contexte dans lequel il est invoqué (contexte fonctionnel ou contexte technique).
- Afin qu'un service s'acquitte au mieux de ses engagements (comportement, fiabilité, performances, …), il doit exercer un contrôle fort sur son environnement d'exécution sous-jacent. Plus ce contrôle est fort, plus l'exécution d'un service est prédictible.

Les aspects d’un service
Autonomie

- Ainsi c’est au moment de la décomposition des fonctions du SI en inventaire de services, qu’il faut définir les membres de ce registre comme des blocs indépendants. C’est donc un haut niveau d’autonomie individuelle des services qui est visé. Réduire l’accès partagé aux ressources d’un service et augmenter le niveau d’isolation physique des services sont deux leviers permettant d’augmenter cette capacité des services à fonctionner de façon autonome.

Les aspects d’un service
Autonomie

Cependant il est quasiment impossible de satisfaire cette caractéristique pour l’ensemble des services, c’est pour cela qu’on a fait introduire deux niveaux basiques d’autonomie :

- **Autonomie de niveau de service :** Les frontières des services entrants dans cette catégorie sont clairement définies et les services sont indépendants les uns des autres, mais il se peut que ces services partagent encore certaines ressources sur lesquelles ils s’appuient.

Les aspects d’un service
Autonomie

- **Autonomie pure :** La logique sous-jacente au service et les ressources qu’il utilise sont la propriété du service et sous son contrôle exclusif. C’est le niveau d’autonomie que l’on pourra atteindre lors de la création de « nouveaux » services quand la logique sous-jacente est construite spécifiquement pour supporter le service. Cependant ceci reste un objectif difficilement atteignable au sein d’un système déjà existant et avec lequel il faut vivre pendant longtemps.

Les aspects d'un service
Sans état (Stateless)

Un service doit minimiser la consommation de ressources en déléguant la gestion des informations d'état quand cela est nécessaire. D'une manière plus générale, la gestion d'états (d'informations de contexte) au sein d'un service pose des problèmes :

- **De compréhension et de maintenabilité**
- **De réutilisation**
- **De performances**

Les aspects d'un service
Sans état (Stateless)

- **De compréhension et de maintenabilité** : La gestion d'états va sensiblement augmenter la complexité de l'implémentation et donc rendre difficile sa lecture, sa documentation, sa testabilité, … Dans le cadre d'une composition de services, la complexité du composé étant directement liée à celle de ses composants, la complexité des services de plus haut niveau peut donc rapidement devenir ingérable.

Les aspects d’un service
Sans état (Stateless)

- **De réutilisation** : La gestion d’états au sein du service brouille la lisibilité de son contrat puisque l’adaptation de son comportement en fonction de son état ne transparaît pas dans son contrat. Or la lisibilité et la transparence des contrats de service sont des facteurs clés de la réutilisation des services. D’autre part, l’utilisation d’états au sein d’un service présuppose souvent l’utilisation de ce service au sein d’un enchaînement d’invocations défini à l’avance. Il sera donc difficile de réutiliser le service au sein d’une autre orchestration ou composition.

Les aspects d'un service
Sans état (Stateless)

- **De performances** : car la gestion des états est consommatrice de ressources systèmes, notamment en terme de stockage de ces états (en mémoire ou sur disque).

On préférera donc la conception et l'implémentation de **services sans état**. La responsabilité de la gestion d'états sera alors déléguée aux utilisateurs (consommateurs) des services : compositions, orchestrations, processus, …

Les aspects d'un service
Découvrabilité

Un service est complété par un ensemble de métas-données de communication au travers desquelles il peut être découvert et interprété de façon effective.

Cette découvrabilité des services existants passe par la mise en œuvre d'un **repository de services** qui vient outiller l'inventaire des services disponibles. Ce repository stocke l'ensemble des métadonnées nécessaires à :

- La recherche des services de l'inventaire.
- La récupération de l'ensemble des artefacts relatifs aux services de l'inventaire (Spécifications, SLAs, Policies, Schémas XML, WSDL, Interfaces, …).

Les aspects d’un service
Découvrabilité

Le concepteur d’un consommateur de service (Service composé, application composite, orchestration, processus, …) pourra s’appuyer sur ce repository de la façon suivante :

- Le concepteur recherche dans le repository un service possédant les fonctionnalités dont il a besoin (1).
- En se basant sur les métadonnées contenues dans le repository, le concepteur est capable de découvrir et d’identifier un service potentiellement capable de répondre à ces besoins (2).
- Le concepteur peut alors accéder au contrat du service : syntaxique, sémantique et de niveau de service (3).

Les aspects d’un service
Composabilité

Un service doit être conçu de façon à participer à des compositions de services.

Ce dernier aspect constitue en quelque sorte l’aboutissement des sept précédents. En effet, l’ensemble des principes présentés dans cette série vise à la réutilisation des services. Or, cette réutilisabilité n’a de sens que si les services sont effectivement réutilisés en prenant part à des compositions de services.

Les aspects d'un service
Composabilité

C'est grâce à la composabilité que sera mis en œuvre le principe de « ***separation of concerns*** » au sein d'une architecture orientée services. L'objectif est ici de déterminer la « bonne » granularité de services afin de décomposer la solution à un problème métier de haut niveau en un ensemble de « plus petites unités réutilisables » de traitement : les services.

L'idée est de pouvoir recomposer notre logique métier à l'infini au sein de processus ou de services composites de haut niveau.

Les aspects d'un service
Composabilité

La mise en œuvre de cet aspect dans une architecture de services pose donc le problème du bon niveau de granularité pour un service :

- Un service trop large ne pourra pas être réutilisé, car il implémente un enchainement de traitements qui n'ont, a priori, de sens que dans le contexte où le service a été écrit. Un service trop large n'est utilisable que par une seule (ou quelques) application(s).
- A l'opposé, un service trop fin ne sera pas réutilisé, car il implémente un traitement atomique qui n'apporte pas de valeur ajoutée. Un service trop fin propose un niveau de détail qui n'est pas pertinent d'un point de vue métier.

Les aspects d’un service

Un service est un composant logiciel distribué satisfaisant les huit aspects (caractéristiques) suivants :

1) Il est exposé à travers un contrat standardisé.
2) Il permet un couplage lâche entre le consommateur et le fournisseur.
3) Il est abstrait (vu comme une boîte noire).
4) Il est réutilisable.
5) Il est autonome.
6) Il est sans état.
7) Il est découvrable (Possibilité de découverte de service).
8) Il est composable (Capacité de composition en modules de services).

Les applications des services Web

Les technologies des services Web peuvent être appliquées à toutes sortes d'applications auxquelles elles offrent des avantages considérables en comparaison aux anciennes API propriétaires, aux implémentations spécifiques à une plate-forme et à quelques autres restrictions classiques que l'on peut rencontrer (multiplateforme, multi-langage, disponible sur Internet avec une information actualisée disponible en temps réel, ...).

Les applications des services Web

L'application des services Web est multiple, autant dans les domaines du B2C, B2B que pour des domaines de gestion, par exemple gestion de stock, gestion commerciale, etc...

- B2C (Business to Consumer) : Qualifie une application, un site Internet destiné au grand public.
- B2B (Business to Business) : Qualifie une application, un site Internet destiné au commerce de professionnel à professionnel.

Applications des Services Web en B2B (Business-to-Business)

- les services web sont principalement utilisés pour **automatiser et fluidifier les processus métier entre entreprises partenaires**.
- L'accent est mis sur l'intégration, l'automatisation et l'efficacité.

B2B Applications Concrètes

Intégration de la Chaîne Logistique (Supply Chain)

- Exemple : Un fabricant (ex: Renault) connecte son système de gestion des stocks (ERP) à ceux de ses fournisseurs de pièces détachées via des services web. Quand le stock d'un composant est bas, le système de Renault commande automatiquement auprès du fournisseur, qui reçoit la commande directement dans son propre système.
- Avantage : Réduction des stocks, livraison juste-à-temps, moins d'erreurs humaines.

Paiements et Facturation Électronique (E-invoicing)

- Exemple : Une entreprise utilise un service web comme PayPal for Business pour intégrer la facturation dans son logiciel de comptabilité. Les factures sont générées, envoyées et les paiements sont enregistrés automatiquement.
- Avantage : Automatisation de la trésorerie, réduction des retards de paiement.

Services Financiers et Bancaires

- Exemple : Une application de gestion financière (comme QuickBooks) se connecte via des API (services web) aux comptes bancaires de l'entreprise pour récupérer automatiquement les transactions et les catégoriser.
- Avantage : Vue financière en temps réel, comptabilité simplifiée.

Logiciel en tant que Service (SaaS) et Interopérabilité

- Exemple : Une entreprise utilise Salesforce (CRM) qui est connectée via des API à son outil de marketing Mailchimp et à son ERP SAP. Quand un nouveau lead est qualifié dans Salesforce, il est automatiquement ajouté à une campagne Mailchimp et créé comme client potentiel dans SAP.
- Avantage : Écosystème logiciel cohérent et données synchronisées.

Réservation et Logistique

- Exemple : Un site de voyage d'affaires se connecte via des services web aux systèmes de réservation (GDS) des compagnies aériennes et des chaînes hôtelières pour afficher la disponibilité et les prix en temps réel.
- Avantage : Accès à un inventaire global, processus de réservation simplifié.

Applications des Services Web en B2C (Business-to-Consumer)

- les services web sont la colonne vertébrale de **l'expérience utilisateur directe**.
- Ils sont utilisés pour fournir des services rapides, personnalisés et engageants aux consommateurs finaux.

B2C Applications Concrètes

E-commerce et Retail

- Exemple : L'application Amazon fait appel à des dizaines de services web en arrière-plan : pour afficher les recommandations de produits, vérifier la disponibilité en stock, calculer les frais de port, et traiter le paiement via son service Amazon Pay.
- Avantage : Expérience d'achat fluide, personnalisée et sécurisée.

Réseaux Sociaux et Partage de Contenu

- Exemple : Quand vous voyez un tweet de Twitter intégré dans un article de presse, c'est un service web de l'API Twitter qui fournit ce contenu. De même, se connecter à un site via son compte Facebook/Google utilise des services web d'authentification (OAuth).
- Avantage : Intégration et partage de contenu sans friction.

Services de Cartographie et de Géolocalisation

- Exemple : Uber utilise les API de Google Maps pour afficher la carte, calculer l'itinéraire optimal et estimer le temps de trajet. L'application de météo sur votre smartphone utilise un service web pour obtenir les données météorologiques en temps réel.
- Avantage : Fonctionnalités complexes externalisées, développement plus rapide.

Streaming de Médias

- Exemple : Netflix et Spotify utilisent des services web pour vous recommander des films ou de la musique basés sur votre historique de visionnage/écoute, gérer votre playlist, et streamer le contenu depuis le cloud.
- Avantage : Expérience hautement personnalisée et adaptive.

Services Bancaires aux Particuliers (Fintech)

- Exemple : Les applications de banque en ligne (ex: Revolut, N26) utilisent des services web pour toutes leurs opérations : consulter son solde, effectuer un virement, ou analyser ses dépenses.
- Avantage : Accès 24/7, interface utilisateur moderne et réactive.

Google+
B2B et B2C

- C’est le futur carrefour d’influence pour les marques et les entreprises adeptes du marketing sur les réseaux sociaux. Google Plus offre des perspectives intéressantes aux professionnels. Par exemple développer votre marque, travailler en mode collaboratif, organiser une veille d’informations.
- Tout cela vous permettra de gagner en présence et communication, présenter votre activité ou entreprise, créer des réseaux internes et externes à l’entreprise, partage d’information, veille concurrentielle…
- Actions pour l’entreprise : Créer un portfolio, créer des cercles à vocation professionnels, chat, vidéo conférences, veille d’informations, réseautage sur mobile lors des déplacements…

Twitter
B2B

- **Twitter** permet d'obtenir des informations brutes, rapides et se tenir au courant de l'actualité des industries**.** Twitter va essentiellement regrouper les personnes qui travaillent dans la publication d'information, l'évènementiel, la publicité, la veille concurrentielle.
- Twitter est un mégaphone qui permet via quelques mots d'accroître la popularité et solliciter les foules en temps réel. Idéal pour créer du trafic autour de l'entreprise.
- Créer une plateforme d'expert : Twitter est un outil d'échanges d'information. L'entreprise est entourée d'experts afin de bénéficier de leur savoir-faire.
- Un tweet c'est viral, puisqu'il se transmet ou renvoie "retweet" en un clic. Donc pour gagner en visibilité, interpeler, les internautes sont motivés de relayer les messages. Puis créer des liens pour gagner en audience…

WordPress
B2B

- **WordPress** communique des informations sérieuses et fiables sur un sujet, et permet de donner son avis.
- Ce site (il s'agit de créer des blogs) s'adresse à des clients qui cherchent à recevoir des informations ou à consulter les services de l'entreprise qu'ils suivent.
- WordPress est réputé pour ses nombreuses fonctionnalités permettant à des utilisateurs, avancés ou non, de créer un site Internet et de le personnaliser. Ses fonctionnalités sont conçues afin de faciliter au maximum l'activité de publication et de la rendre accessible à tous.

Viadeo et LinkedIn
B2B

- **Viadeo et LinkedIn** permettent de construire un réseau solide de votre activité. Les deux réseaux permettent de cibler, respectivement à l'échelle française et internationale.
- Ces réseaux du **B2B** sont des portails très intéressants qui communiquent entre le monde de l'emploi des formations et de l'entreprise. Ils sont des plateformes d'échange entre candidats, étudiants, entreprises, recruteurs.

Facebook et Myspace
B2C

- Facebook et Myspace sont des réseaux sociaux.
- Ces sites sont majoritairement utilisés pour rester en contact avec des amis ou fans, pour communiquer, partager des photos, des liens ou des vidéos ou encore échanger des informations sur divers centres d’intérêt.

Types de Services Web

L’architecture fonctionnelle des services web utilise plusieurs langages et protocoles durant le déploiement et l’invocation des services web. Les services web se basent sur l’utilisation des normes actuelles d'Internet comme le protocole HTTP.

Types de Services Web sont :

- SOAP
- REST
- gRPC

QCM

- 20 questions
- Lien : https://docs.google.com/forms/d/1Iu4K7ByEu57RANjMz5_4Y7UpEoVESxZ1nviW2rBk3E4/prefill

Vidéo

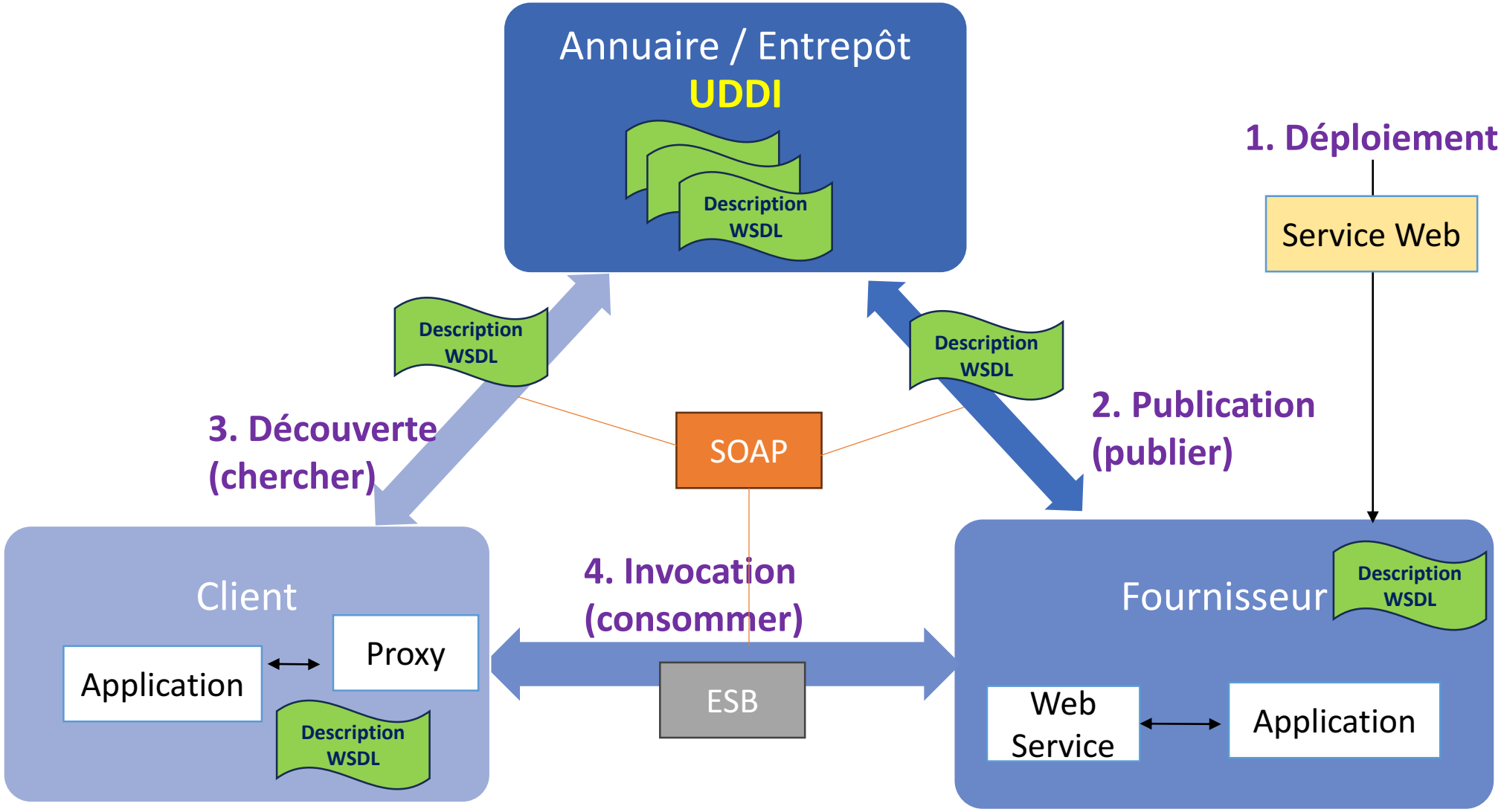
- https://youtu.be/Vnk0p-63mHQ
- https://youtu.be/gBmEo_Lw_KI
- https://youtu.be/V01GwRMCk2s

Discussion



Chapitre 3 : Web Services SOAP

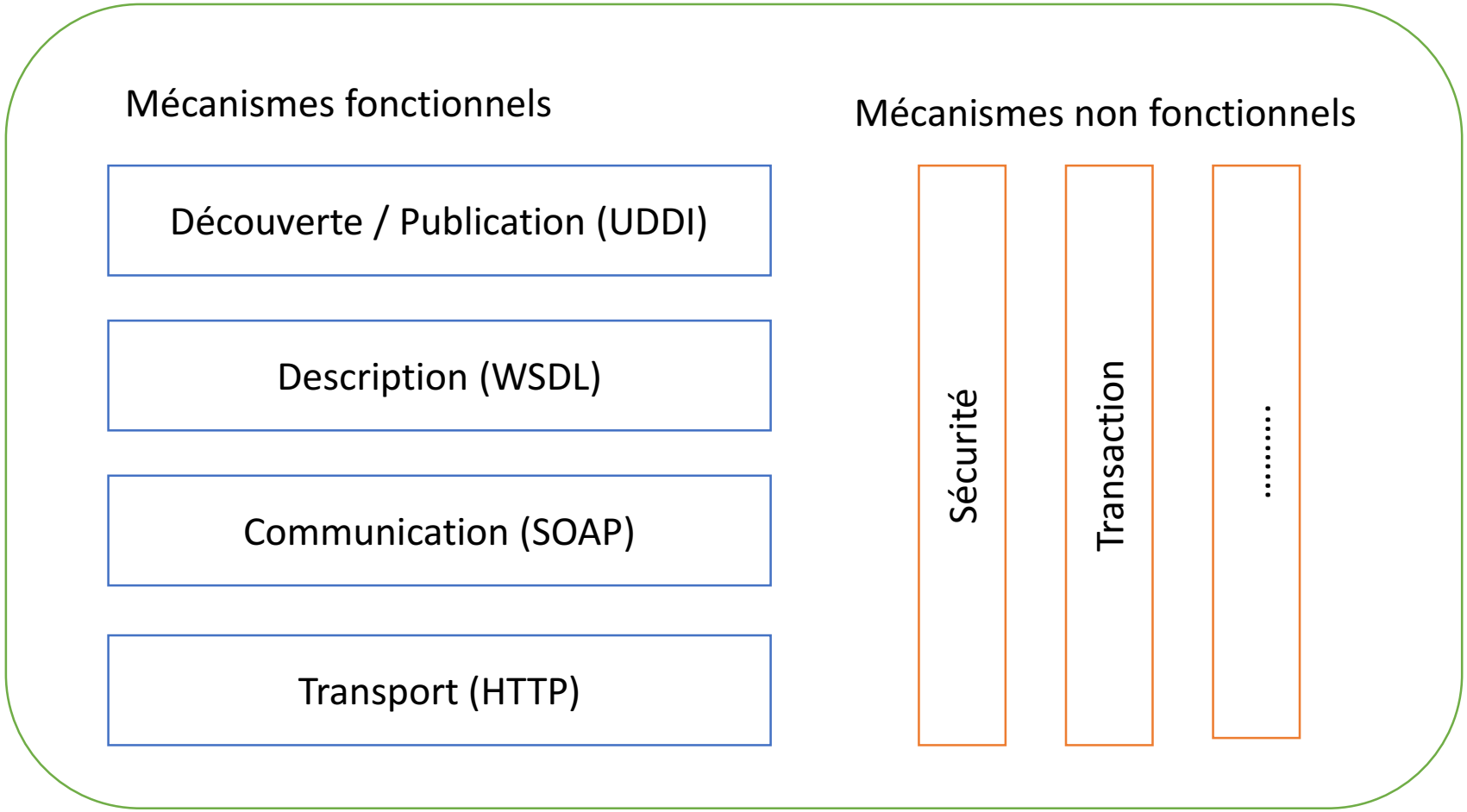
Technologies



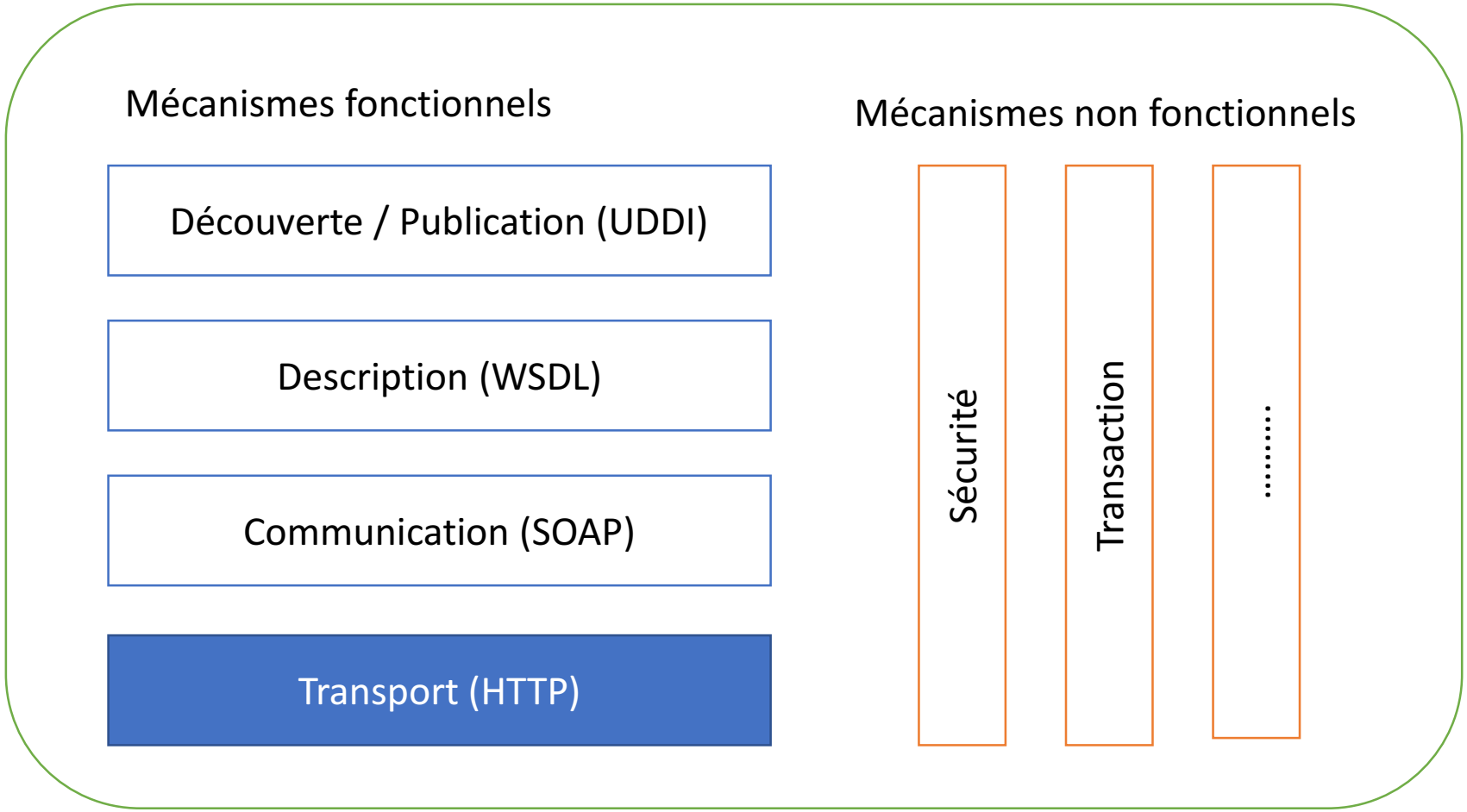
Technologies

- Contrat / description WSDL (Web Service Description Language) : contient la description du service et de son utilisation, échangée entre le fournisseur de service et le client.
- SOAP(Simple Object Access Protocol) : Protocole de communication au sein de la SOA
- Annuaire UDDI (Universal Description Discovery and Integration) : liste dynamique de recherche de descriptions de services dans laquelle les fournisseurs de services publient leurs descriptions.
- Bus ESB (Enterprise Service Bus) : le moyen par lequel s'effectue la communication entre le client et le fournisseur du service. Son but est avant tout de permettre la communication des applications qui à la base ne sont pas pensées pour fonctionner ensemble.

La pile protocolaire simplifiée SOAP WS



La pile protocolaire simplifiée SOAP WS



La couche de Transport HTTP (Hypertext Transfer Protocol)

- Cette couche s'intéresse aux protocoles de transport de bas niveau, ces derniers vont transporter les requêtes et les réponses échangées entre services. HTTPS est la variante sécurisée de ce protocole.
- Le protocole le plus utilisé et recommandé par le consortium WS-I (Web Service Interoperability) est l'HTTP, mais d'autres implémentations peuvent utiliser un autre ensemble de protocoles tels que : FTP (File Transfer Protocol), JMS (java messagerie services), SMTP (Simple Mail Transfer Protocol), etc.

La couche de Transport HTTP (Hypertext Transfer Protocol)

Le protocole HTTP, à travers les Cookies, fournit un moyen pour "marquer" l'agent client de façon suffisamment précise :

- L'application est marquée, puisque c'est dans le container de cette application que l'information de marquage est stockée.
- L'utilisateur d'une machine multi-utilisateur est marqué, puisque les informations de marquage sont stockées dans le profil particulier de cet utilisateur.

La couche de Transport HTTP (Hypertext Transfer Protocol)

Structure Du HTTP :

Notion d'entité : Le HTTP est un protocole destiné à transporter des entités document. Il est constitué de messages-requêtes (du client au serveur) et de messages-réponse (du serveur au client). Il implémente donc parfaitement un paradigme client-serveur en ce sens que :

- Le client (agent utilisateur) est toujours l'initiateur de la requête (PULL).
- Le serveur est toujours en attente d'un client.

La couche de Transport HTTP (Hypertext Transfer Protocol)

Structure Du HTTP :

Les messages transportent des entités. Le cas le plus fréquent est celui d'une entité unique. L'entité est toujours définie par :

- Une entête d'entité (méta informations sur le contenu et la transmission).
- Un corps d'entité (le contenu).

La couche de Transport HTTP (Hypertext Transfer Protocol)

Structure Du HTTP :

Le protocole admet cependant que plusieurs entités puissent être transportées par le même message.

C'est le cas du téléchargement de fichiers accompagnant des données de formulaire, ou de téléchargements multiples.

Dans ce cas, le message doit permettre de séparer proprement les différentes entités qu'il transporte.

La couche de Transport HTTP (Hypertext Transfer Protocol)

Construction De La Requête

1. Le libellé de requête : La requête HTTP, comme son nom l'indique est une demande à un serveur pour qu'il exécute un ordre (principalement d'obtenir un document). Ces ordres sont symbolisés par des verbes.

Les verbes les plus connus sont :

- **HEAD** : "donne-moi les métas informations concernant un document" ;
- **GET** : "donne-moi le contenu du document (et ses méta informations par la même occasion)" ;
- **POST** : "fais ce que tu veux (ou tu peux avec les données que je t'envoie)" ;
- **PUT** : "mets le document que je t'envoie où je te le dis".

La couche de Transport HTTP (Hypertext Transfer Protocol)

2. La Cible : Elle est représentée par une URL absolue ou relative. Le serveur est chargé de "consommer" cette cible, la traduire en une réalité physique tangible (un fichier physique, un exécutable, un répertoire).

La requête peut informer le serveur d'un certain nombre de prédispositions de l'agent utilisateur.

Des mécanismes de gestion de préférences, convenues entre le client et le serveur permettent d'améliorer le service rendu, sans faire appel à des choix explicites de l'utilisateur.

La couche de Transport HTTP (Hypertext Transfer Protocol)

Ces préférences sont, par exemple :

- Le choix de la langue la plus appropriée.
- Le choix d'un encodage acceptable du côté du client.
- Le choix d'une version suffisamment récente, ou au contraire de versions plus anciennes.

Les champs de requête sont placés à la suite de la "ligne VERBE" sous la syntaxe :

NomChamp : [ValeurChamp] LF

L'en-tête doit se terminer par une ligne vide, pour indiquer que ce qui suit est un corps d'entité (ou que le message est fini).

La couche de Transport HTTP (Hypertext Transfer Protocol)

3. Un corps de requête

- Certains verbes (PUT, POST) supposent très fortement une transmission de contenu du client vers le serveur.
- Ce contenu pouvant être a priori d'une taille quelconque, il s'agit bien d'un corps d'entité.
- Dans ce cas, on doit considérer potentiellement une requête comme étant une transmission d'un document à part entière.
- Le client pousse un document local vers le serveur.

La couche de Transport HTTP (Hypertext Transfer Protocol)

4. Détermination de la longueur du corps d'entité

- Il n'existe pas en HTTP de bloc de fin de message identifiable.
- D'autre part, virtuellement, le corps de l'entité peut contenir n'importe quelle séquence binaire, y compris une séquence qui pourrait se confondre avec un tel bloc.
- La question de prévoir la fin du corps d'entité doit être réglée dès l'entête.
- Toute requête portant un contenu d'une certaine longueur doit informer le serveur de cette longueur. On utilise le champ : Content-Length

La couche de Transport HTTP (Hypertext Transfer Protocol)

Construction De La Réponse

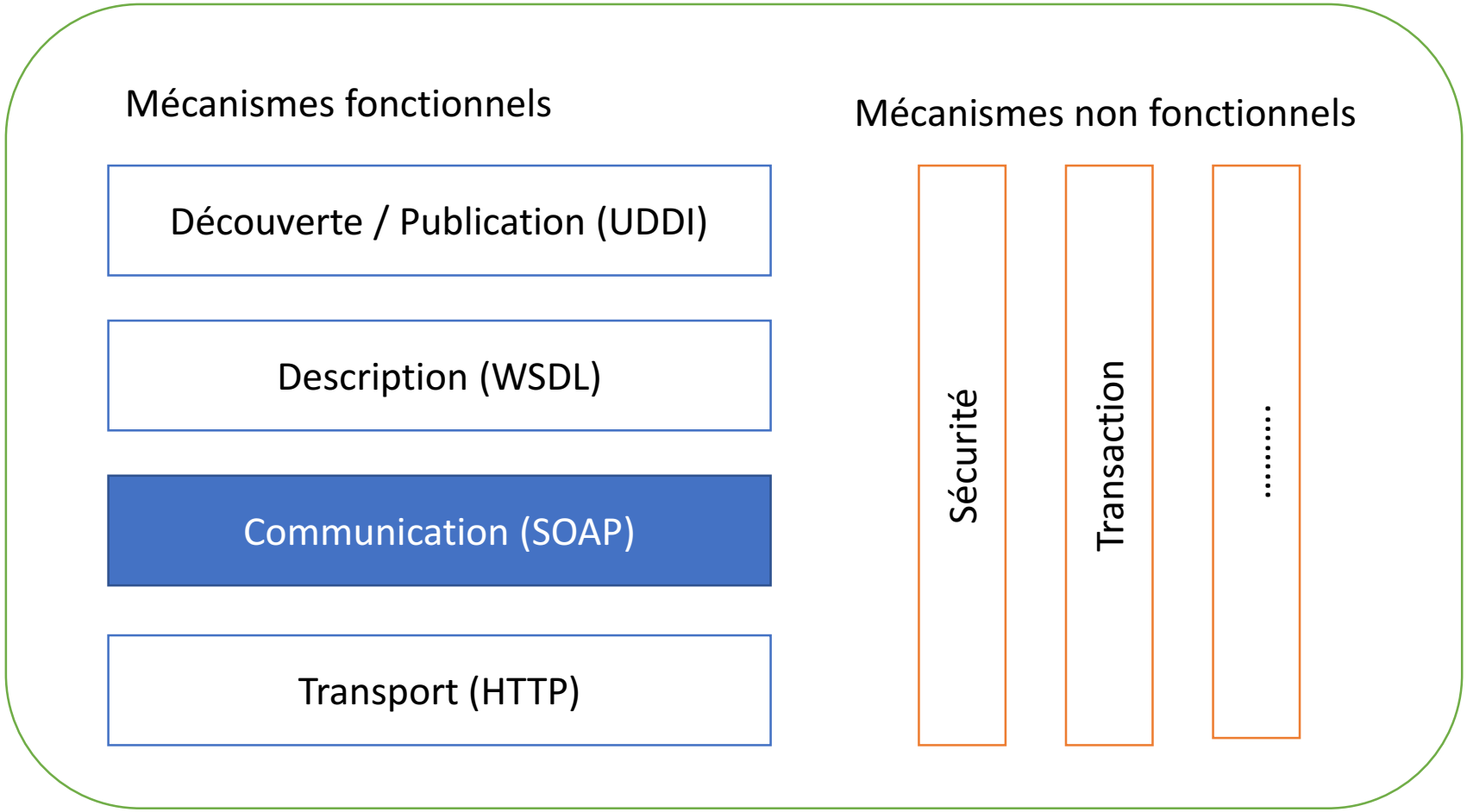
Le schéma de réponse est symétrique à celui de la requête, à la différence près que le corps d'entité est le plus souvent non vide.

La couche de Transport HTTP (Hypertext Transfer Protocol)

- Une requête mono-entité, dans le cas général se construit ainsi :

```
[verbe] [Url] [version HTTP] LF
Content-Length : [longueur du corps]
[Nom Champ]:[Valeur Champ] LF
...
LF (sépare l'en-tête et le corps d'entité)
[Octets de l'entité]
```

La pile protocolaire simplifiée SOAP WS



La couche de Communication SOAP (Simple Object Access Protocol)

Définition:

- SOAP (***Simple Object Access Protocol***) est un protocole standardisé par W3C, et qui assure le transfert des messages et les appels de procédures RPC (Appels de procédures distants) indépendamment du transport.
- SOAP définit le cadre général pour l'échange de données structurées en **XML**.
- Il permet d'échanger des structures de données complexes en XML avec les espaces de nom, et la spécification XML Schéma.
- L'atout principal des protocoles SOAP et UDDI ainsi que du langage WSDL est de se reposer sur le langage XML (eXtensible Markup Language).

C’est quoi XML ?

Le langage XML (eXtensible Markup Language)

- XML est un standard qui permet de décrire des documents structurés transportables sur les protocoles d'Internet.
- Il apporte à l'architecture des services web l'extensibilité et la neutralité vis-à-vis des plateformes et des langages de développement.
- De plus, grâce à la structuration, XML permet la distinction entre les données des applications et les données des protocoles.

Le langage XML (eXtensible Markup Language)

- La technologie des services web a été conçue pour fonctionner dans des environnements totalement hétérogènes.
- Cependant, l'interopérabilité entre les systèmes hétérogènes demande des mécanismes puissants de correspondance et de gestion des types de données des messages entre les différents participants (clients et fournisseurs).
- C'est une tâche où les schémas de type de données XML s'avèrent bien adaptés. C'est pour cette raison que la technologie des services web est essentiellement basée sur XML.

Le langage XML (eXtensible Markup Language)

Quelques spécifications de XML :

- **XSD (XML Schema)** : c'est un langage qui sert à décrire formellement un vocabulaire
- **XSLT (Extensible Stylesheet Language Transformations)** : est utilisé pour transformer un document XML basé sur un certain schéma en un autre document XML qui peut être un document lui-même basé sur un autre schéma.
- **XPath (XML Path Language)** : fournit une syntaxe d'expressions utilisées pour créer des chemins de localisation.

Le langage XML (eXtensible Markup Language)

- **Espaces de noms** : Mécanismes permettant de partitionner les balises XML (permet d'avoir deux fois le même nom de balise)

Un espace de nom est défini dans n'importe quelle balise par l'attribut xmlns et par une URI.

Dans un document XML, un espace de noms est identifié par un nom logique, les balises appartenant à cet espace doivent alors être préfixée par ce nom logique.

Exemple :

<meta:body xmlns:meta="http://meta.lip6.fr/meta/"

Le langage XML (eXtensible Markup Language)

Exemple d’un fichier XML:

```
<livre>
        <titre> le super livre </titre>
        <chapitre>
                <numero> 1 </numero>
                <titre> titre du chapitre 1 </titre>
                <contenu> xxxxxxxxx </contenu>
        </chapitre>
        <chapitre>
                …
        </chapitre>

</livre>
```

Le langage XML (eXtensible Markup Language)

Deux façons de définir une **grammaire** XML :

- DTD
  - Langage de définition de grammaire XML
  - Largement utilisé
  - Expression faible (type, structure)
- XML Schéma
  - Langage XML de définition de grammaire XML
  - De plus en plus utilisé
  - Expression puissante (**type**, structure, héritage)

Un document XML est dit **valide** lorsqu'il est **conforme** à une grammaire.

Le langage XML (eXtensible Markup Language)

Avantages du langage XML :

- Standard W3C
- La syntaxe XML ne contient que peu de mot clef: Simplicité
- XML est indépendant des plates-formes: Portabilité
- XML est un méta-langage, il est possible de créer ses propres balises: Extensibilité
- Outils disponibles (et gratuits)
- Largement utilisé pour les échanges inter-applications

La couche de Communication SOAP (Simple Object Access Protocol)

Définition:

- SOAP (***Simple Object Access Protocol***) est un protocole standardisé par W3C, et qui assure le transfert des messages et les appels de procédures RPC (Appels de procédures distants) indépendamment du transport.
- SOAP définit le cadre général pour l'échange de données structurées en **XML**.
- Il permet d'échanger des structures de données complexes en XML avec les espaces de nom, et la spécification XML Schéma.

La couche de Communication SOAP (Simple Object Access Protocol)

Objectif : permettre la normalisation des échanges de données.

Il définit un moyen uniforme d'échange de données encodées XML sous forme d'appels de procédures à distance RPC, en utilisant souvent le HTTP comme protocole de transport ainsi les protocoles SMTP, POP, et FTP dans certains cas.

Il assure l'interopérabilité entre composants tout en restant indépendant des plates-formes et des langages de programmation.

La couche de Communication SOAP (Simple Object Access Protocol)

Il repose sur deux standards :

- XML : pour la structuration des messages.
- http : pour le transport.

Il permet une interopérabilité avec divers environnements logiciels (.NET, Java/RMI, CORBA, COM/DCOM …) quel que soit leur plate-forme d'exécution.

La couche de Communication SOAP (Simple Object Access Protocol)

- Les paquets de données circulent sous format XML, mais tous types de données peuvent être transportés même si les données binaires peuvent faire l'objet d'un encodage spécifique.
- Les données ayant un format binaire sont transportées sous forme de pièces jointes avec le message SOAP principal.
- Les messages SOAP peuvent traverser les Proxy et les pare-feu.
- Il permet l'échange de données que ce soit en mode synchrone (requête/réponse) ou asynchrone (accompagnement des processus).

La couche de Communication SOAP (Simple Object Access Protocol)

PRINCIPES FONDAMENTAUX DU PROTOCOLE SOAP :
1. Framework de messagerie
2. Encodage et sérialisation standard pour les objets
3. Invocation de Service d'objets distants via SOAP RPC
4. Messages SOAP
5. Acheminement des messages
6. Nœud intermédiaire actif
7. Règles d'encodage
8. Liaison du SOAP avec les protocoles du transport
9. Sécurité du SOAP

La couche de Communication SOAP (Simple Object Access Protocol)

1. Framework de messagerie :

Concepts relatifs spécifiquement à la manière avec laquelle les messages sont manipulés.

Nœud SOAP (Nodes) : Un nœud SOAP (identifié par un URI) est l'infrastructure technique qui alimente les différents scénarios de communication entre services. Il peut être vu comme étant le processeur logique permettant de recevoir, transmettre, et exécuter un ensemble d'opérations sur les messages SOAP.

Différents rôles joués par un nœud :

- Expéditeur initial (Demandeur de service)(***Initial Sender***) ou (***Service Requestor***).
- Récepteur Final (Fournisseur de service) (***Ultimate Receiver***) ou (***Service Provider***).
- Nœud intermédiaire (***Intermediary***)

La couche de Communication SOAP (Simple Object Access Protocol)

2. Encodage et sérialisation standard pour les objets:

- Le protocole SOAP impose une spécification d'encodage/sérialisation standard, qui doit être appliquée sur les données du message, que ce soit du côté client ou bien fournisseur. Cette tâche est assurée par le runtime SOAP implanté des deux côtés (client, fournisseur), et essentiellement utilisée avec le RPC.
- Exemple: encodage d'un tableau d'entiers :

```
<SOAP-ENC:Array SOAP-ENC:ArrayType="xsd:int[3]">
        < SOAP-ENC:int> 8 </ SOAP-ENC:int>
        <SOAP-ENC:int> 8 </ SOAP-ENC:int>
        <SOAP-ENC:int> 8 </ SOAP-ENC:int>
</SOAP-ENC:Array>
```

La couche de Communication SOAP (Simple Object Access Protocol)

3. Invocation de Service d'objets distants via SOAP RPC :

Le protocole SOAP permet d'invoquer un objet distant en communiquant les informations nécessaires à l'appel: nom de la méthode, et sa signature numérique (ses arguments) dans un message au format XML. La réponse à la requête est aussi renvoyée encapsulée dans un message au format XML. La librairie SOAP assure l'envoi du RPC à l'aide du protocole du transport choisi.

La couche de Communication SOAP (Simple Object Access Protocol)

Les différentes étapes d'invocation des objets distants sont:

1. Le programme client SOAP crée le document XML contenant les informations nécessaires à l'invocation d'un objet distant, et le transmet au protocole HTTP (requête POST).
2. Le message est transmis au serveur via une connexion HTTP.
3. Le serveur reçoit le message et fait appel à l'objet concerné.
4. L'objet effectue le traitement demandé et renvoie les résultats au serveur SOAP.
5. Le serveur SOAP encapsule les résultats et les envoie dans un document XML au client via le protocole HTTP.
6. Le client SOAP décode le message et envoie les résultats au demandeur initial.

La couche de Communication SOAP (Simple Object Access Protocol)

- **Exemple:** appel du service GetLastTradePrice:

```
Post /StockQuote HTTP/1.1
Host : www.stockquoteserver.com
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn
SOAPAction: "http://example.org/2001/06/quotes"
<env:Envelope xmlns:env="http://www.w3.org/2001/06/soap-envelope">
    <env:Body>
        <m: GetLastTradePrice
        Env:encodingStyle=="http://www.w3.org/2001/06/soap-
        encoding"
        Xmlns:m="http://example.org/2001/06/quotes">
            <symbol>DIS</symbol>
        </m: GetLastTradePrice>
    </env:Body>
</env:Envelope
```

La couche de Communication SOAP (Simple Object Access Protocol)

4. Messages SOAP :

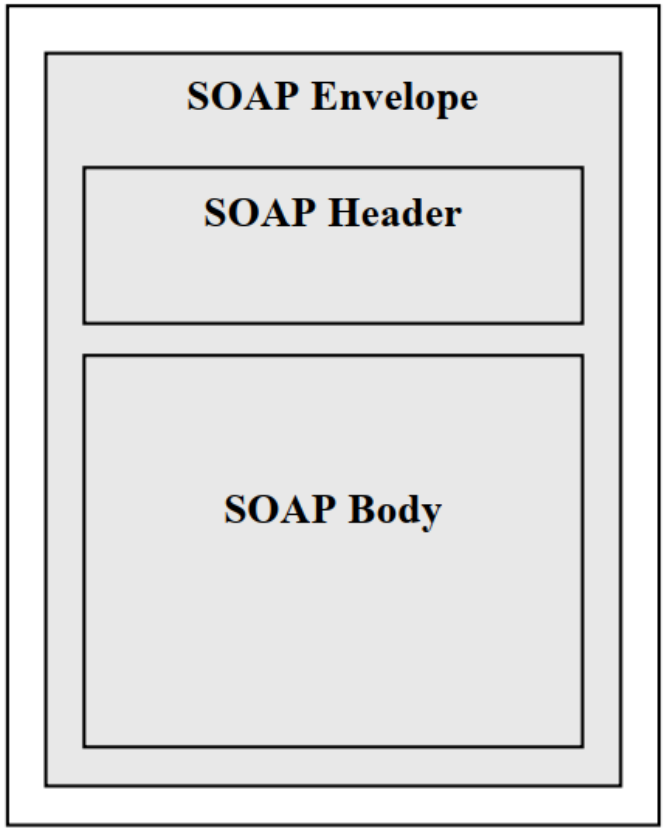
- Le framework de messagerie SOAP demande que les messages SOAP soient composés d'une enveloppe (***Envelope***) contenant un entête (***Header***) et un corps (***Body***). La partie entête du message qui sert à véhiculer les méta-données du message est une partie optionnelle. Elle est formée d'un ensemble de blocs appelés "entrées entête", dont chacun peut contenir plusieurs attributs.
- La partie Body est la partie principale du message, elle contient les données applicatives destinées au récepteur final.
- Cependant un message SOAP peut avoir des fichiers attachés au fichier principal, ainsi tout fichier qui n'est pas au format xml doit apparaître dans la partie attachement. Ainsi tous les fichiers attachés en pièces jointes doivent être référencés par des URL dans le message principal.

La couche de Communication SOAP (Simple Object Access Protocol)

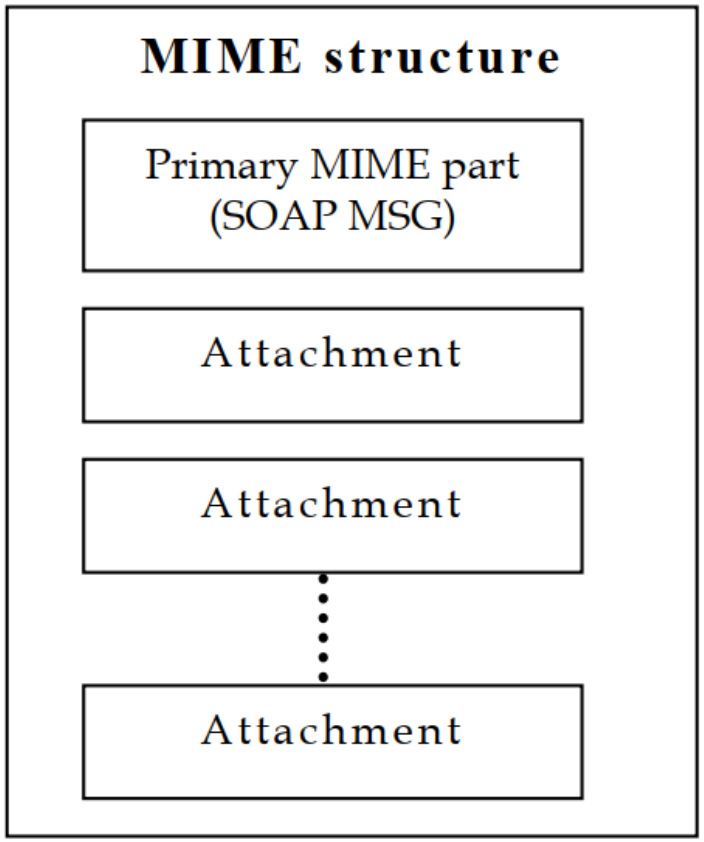
- **Exemple :** (Squelette d'un message SOAP)

```
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header>
    ...
  </env:Header>
  <env:Body>
    ...
  </env:Body>
</env:Envelope>
```

La couche de Communication SOAP (Simple Object Access Protocol)

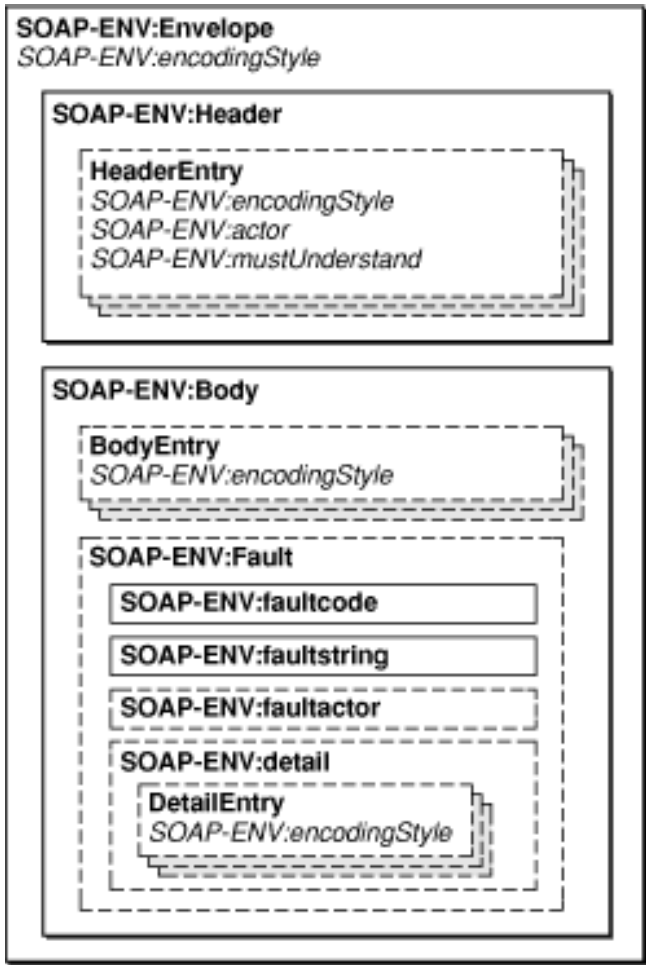


Message SOAP simple



Message SOAP avec pièces jointes

La couche de Communication SOAP (Simple Object Access Protocol)



La couche de Communication SOAP (Simple Object Access Protocol)

5. Acheminement des messages :

A l'arrivée d'un message chez un nœud SOAP, celui-ci doit :

1. Vérifier la structure du message: enveloppe, entête, et corps du message.
2. Déterminer les entrées d'entêtes destinées à lui-même, et l'ensemble des rôles à en exécuter.
3. Sélectionner parmi ces entrées celles qui sont mandataires.
4. Si une de ces entrées n'est pas comprise ou bien qu'il n'a pas la capacité d'en traiter, il envoie un message d'erreur à l'expéditeur initial.
5. Traiter toutes les entrées mandataires, ainsi que celles qui sont optionnelles et dont il a la capacité d'en traiter.
6. Traiter le corps du message s'il s'agit du récepteur final.
7. Retransmettre le message au nœud suivant s'il s'agit d'un nœud intermédiaire.

La couche de Communication SOAP (Simple Object Access Protocol)

6. Nœud intermédiaire actif :

En plus du traitement effectué par des intermédiaires de réacheminement, les intermédiaires actifs entreprennent du traitement supplémentaire pouvant modifier le message sortant selon des manières non décrites dans le message entrant. C'est-à-dire qu'ils peuvent entreprendre un traitement non décrit par un bloc d'en-tête SOAP du message arrivé. L'ensemble potentiel de services fournis par un intermédiaire actif inclut : des services de sécurité, des services d'annotation et des services de manipulation de contenu.

La couche de Communication SOAP (Simple Object Access Protocol)

7. Règles d'encodage:

Les règles d'encodage SOAP expriment les types de données définis pour l'application en XML. Elles constituent un ensemble unique de règles d'encodage défini par la spécification SOAP. Cet encodage est fondé sur les schémas XML W3C. Il existe deux types de données :

a) Les types de données simples : Chaînes de caractères, Enumérations, Tableaux d'octets

b) Les types composés :

- ***Struct*** (enregistrement) qui désigne une valeur composée dans laquelle le nom d'accesseur seul permet de différencier les valeurs de membres, chaque accesseur ayant un nom unique.

- ***Array*** qui désigne un tableau dont les éléments sont de même type.

La couche de Communication SOAP (Simple Object Access Protocol)

8. Liaison du SOAP avec les protocoles du transport:

a) Modes de transport de message SOAP: Les messages sont véhiculés soit en mode RPC ou bien en mode messagerie.

b) Utilisation de SOAP avec RPC:

Le protocole SOAP **encapsule** et permet d'échanger les appels RPC en utilisant la flexibilité du XML. Les appels et réponses de méthodes RPC sont transmis **encapsulés** dans l'élément Body du message. Pour effectuer une invocation de méthode distante les informations suivantes sont nécessaires: URI de l'objet cible, Nom de la méthode, Signature de la méthode (facultative), Paramètres de la méthode et Données de l'entête (facultatives).

La couche de Communication SOAP (Simple Object Access Protocol)

9. Sécurité du SOAP:

La structure de messages SOAP ne fournit pas directement de mécanismes pour traiter le contrôle d'accès, la confidentialité, l'intégrité et la non répudiation. De tels mécanismes peuvent être fournis sous forme d'extensions SOAP grâce au modèle d'extensibilité.

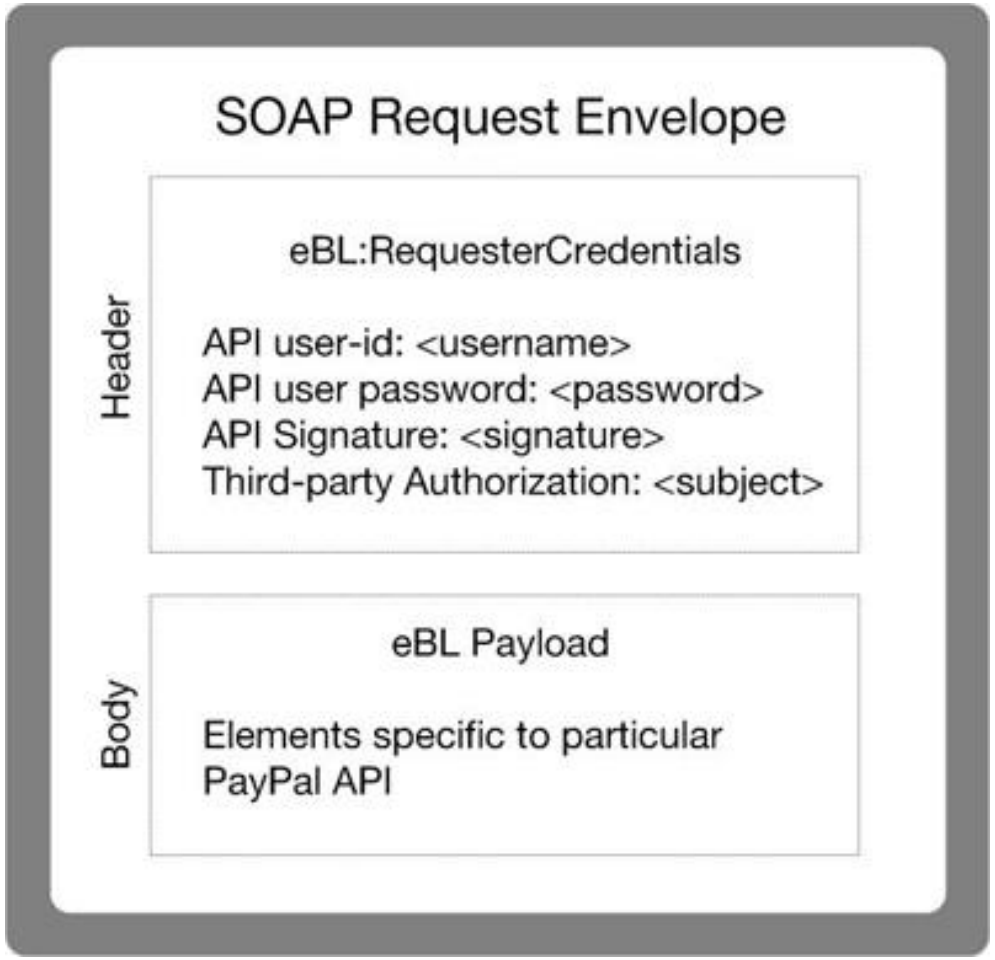
Le concepteur et l'implémenteur du protocole SOAP doivent prendre en considération différents types de risques qui peuvent être produits :

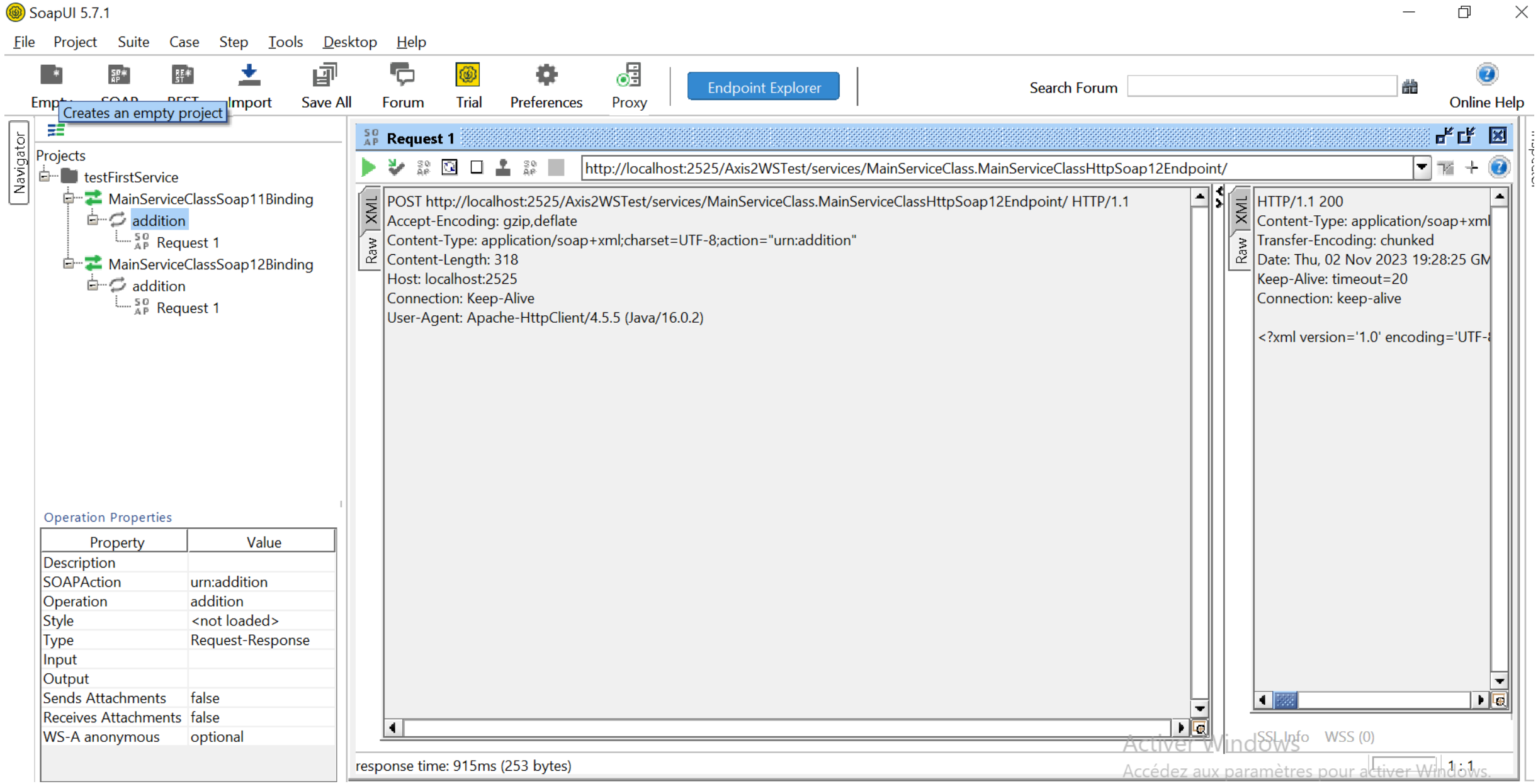
a) **Risque dû au contenu du message (nœud SOAP)**

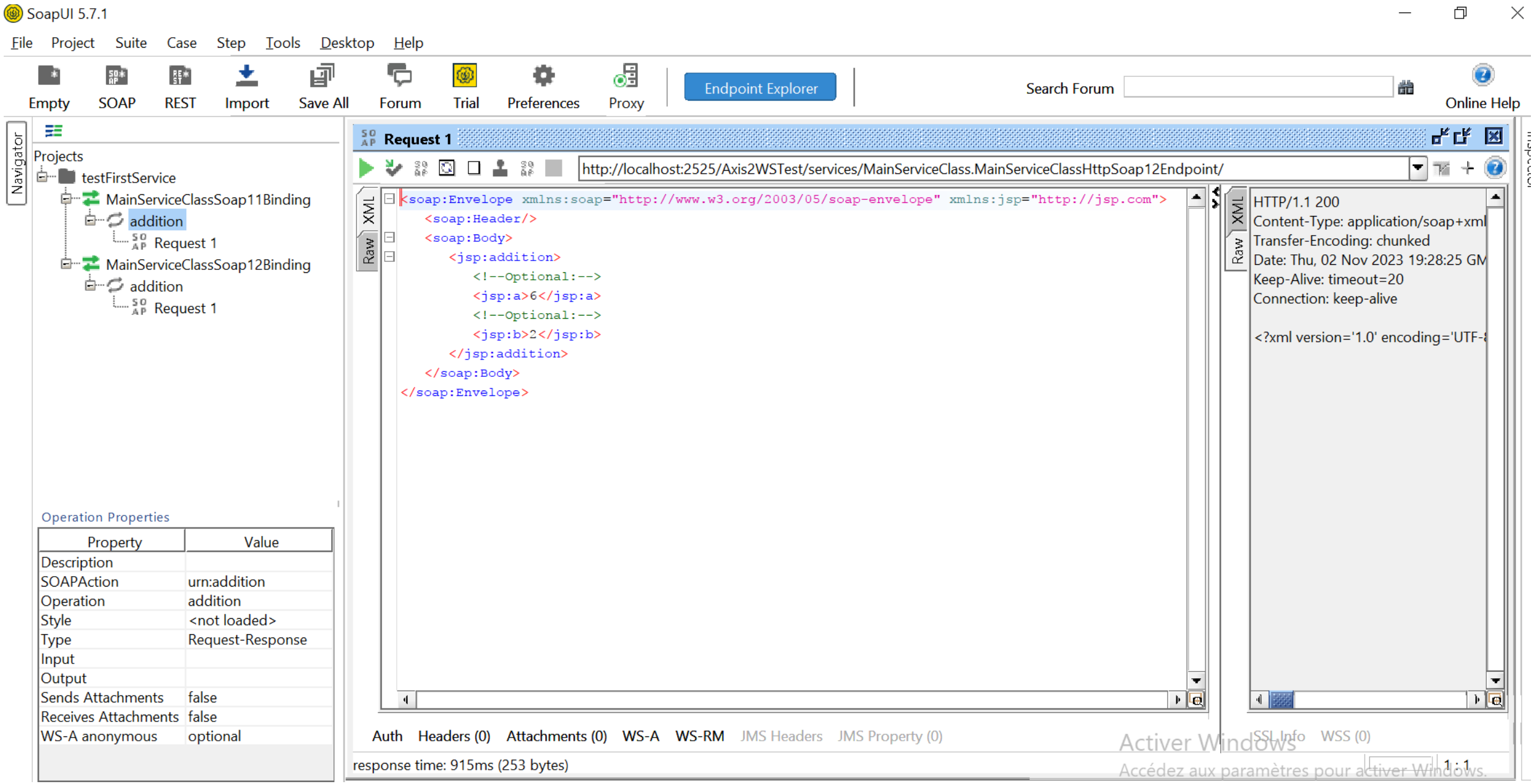
b) **Risque dû au transport (nœuds intermédiaires)**

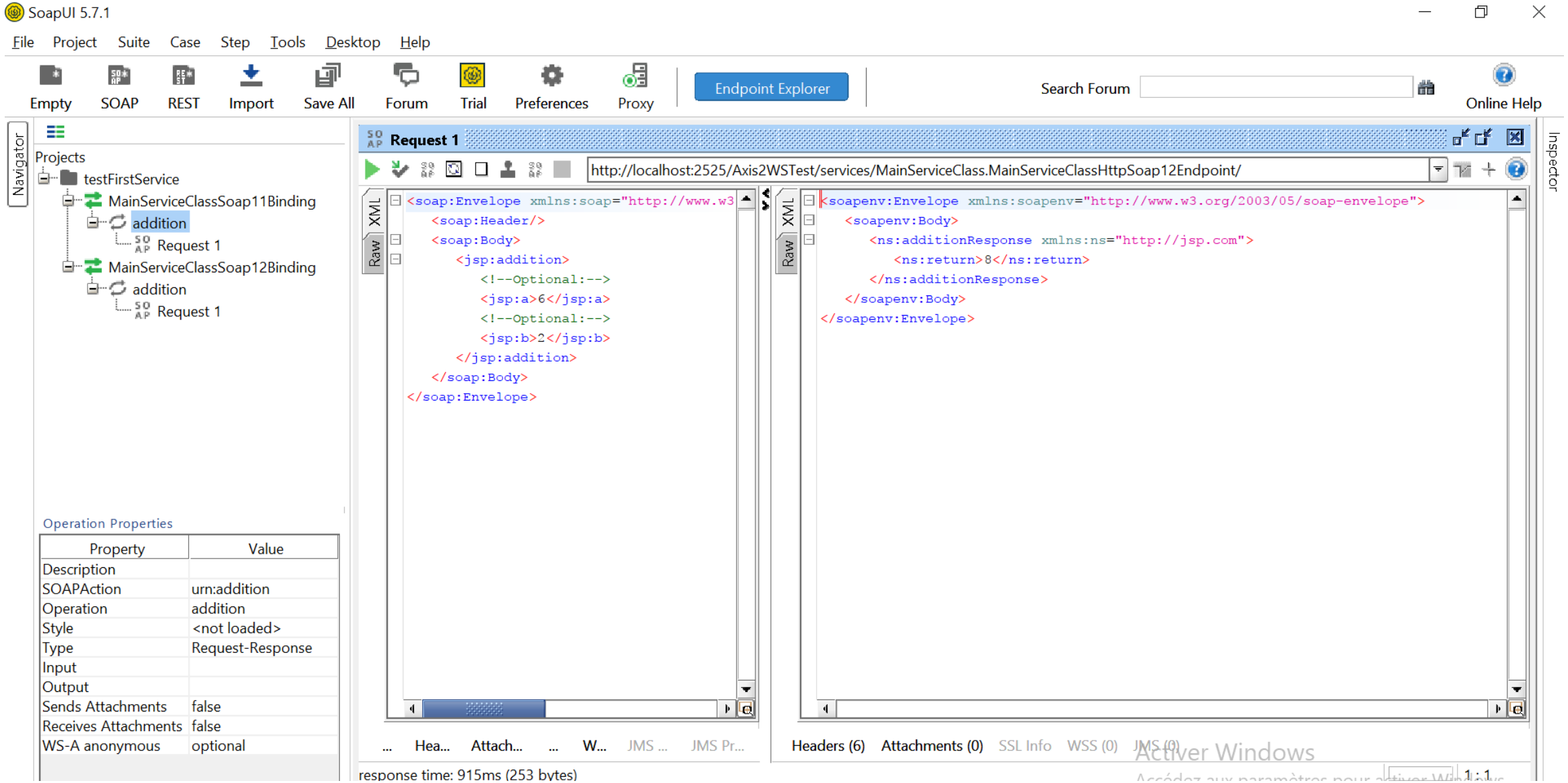
c) **Risque dû aux spécifications (protocole de transport)**

Exemple


SOAP Request Envelope
Header
eBL:RequesterCredentials
API user-id: <username>
API user password: <password>
API Signature: <signature>
Third-party Authorization: <subject>
Body
eBL Payload
Elements specific to particular
PayPal API


SoapUI 5.7.1
File Project Suite Case Step Tools Desktop Help
Creates an empty project
Import
Save All
Forum
Trial
Preferences
Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
Projects
testFirstService
MainServiceClassSoap11Binding
addition
Request 1
MainServiceClassSoap12Binding
addition
Request 1
Operation Properties
Property
Value
Description
SOAPAction
urn:addition
Operation
addition
Style
<not loaded>
Type
Request-Response
Input
Output
Sends Attachments
false
Receives Attachments
false
WS-A anonymous
optional
Request 1
http://localhost:2525/Axis2WSTest/services/MainServiceClass.MainServiceClassHttpSoap12Endpoint/
XML
Raw
POST http://localhost:2525/Axis2WSTest/services/MainServiceClass.MainServiceClassHttpSoap12Endpoint/ HTTP/1.1
Accept-Encoding: gzip,deflate
Content-Type: application/soap+xml;charset=UTF-8;action="urn:addition"
Content-Length: 318
Host: localhost:2525
Connection: Keep-Alive
User-Agent: Apache-HttpClient/4.5.5 (Java/16.0.2)
HTTP/1.1 200
Content-Type: application/soap+xml
Transfer-Encoding: chunked
Date: Thu, 02 Nov 2023 19:28:25 GM
Keep-Alive: timeout=20
Connection: keep-alive
<?xml version='1.0' encoding='UTF-
SSL Info
WSS (0)
response time: 915ms (253 bytes)
1 : 1
Activer Windows
Accédez aux paramètres pour activer Windows.


SoapUI 5.7.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
Projects
testFirstService
MainServiceClassSoap11Binding
addition
Request 1
MainServiceClassSoap12Binding
addition
Request 1
Operation Properties
Property Value
Description
SOAPAction urn:addition
Operation addition
Style <not loaded>
Type Request-Response
Input
Output
Sends Attachments false
Receives Attachments false
WS-A anonymous optional
Request 1
http://localhost:2525/Axis2WSTest/services/MainServiceClass.MainServiceClassHttpSoap12Endpoint/
XML
Raw
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope" xmlns:jsp="http://jsp.com">
<soap:Header/>
<soap:Body>
<jsp:addition>
<!--Optional:-->
<jsp:a>6</jsp:a>
<!--Optional:-->
<jsp:b>2</jsp:b>
</jsp:addition>
</soap:Body>
</soap:Envelope>
HTTP/1.1 200
Content-Type: application/soap+xml
Transfer-Encoding: chunked
Date: Thu, 02 Nov 2023 19:28:25 GM
Keep-Alive: timeout=20
Connection: keep-alive
<?xml version='1.0' encoding='UTF-
Auth Headers (0) Attachments (0) WS-A WS-RM JMS Headers JMS Property (0)
SSL Info WSS (0)
response time: 915ms (253 bytes)
1 : 1
Activer Windows
Accédez aux paramètres pour activer Windows.


SoapUI 5.7.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
Projects
testFirstService
MainServiceClassSoap11Binding
addition
Request 1
MainServiceClassSoap12Binding
addition
Request 1
Operation Properties
Property Value
Description
SOAPAction urn:addition
Operation addition
Style <not loaded>
Type Request-Response
Input
Output
Sends Attachments false
Receives Attachments false
WS-A anonymous optional
Request 1
http://localhost:2525/Axis2WSTest/services/MainServiceClass.MainServiceClassHttpSoap12Endpoint/
XML
Raw
<soap:Envelope xmlns:soap="http://www.w3
<soap:Header/>
<soap:Body>
<jsp:addition>
<!--Optional:-->
<jsp:a>6</jsp:a>
<!--Optional:-->
<jsp:b>2</jsp:b>
</jsp:addition>
</soap:Body>
</soap:Envelope>
<soapenv:Envelope xmlns:soapenv="http://www.w3.org/2003/05/soap-envelope">
<soapenv:Body>
<ns:additionResponse xmlns:ns="http://jsp.com">
<ns:return>8</ns:return>
</ns:additionResponse>
</soapenv:Body>
</soapenv:Envelope>
Hea... Attach... W... JMS ... JMS Pr...
Headers (6) Attachments (0) SSL Info WSS (0) JMS (0)
response time: 915ms (253 bytes)
Activer Windows
Inspector

La couche de Communication SOAP (Simple Object Access Protocol)

IMPLEMENTATION SOAP (Voir TP1)

Une multitude d'implémentations du protocole SOAP et Toolkits existent aujourd'hui, qui correspondent aux différents environnements et langages de programmation (Java, C++, C, Perl, PHP, Python..) et qui peuvent répondre aux différents besoins de l'utilisateur.

Ces outils peuvent implémenter de différentes fonctionnalités, au-delà des fonctionnalités de base communes:

- Processus fondamental de création, déploiement et utilisation des services web.
- Comprendre et réaliser tous les traitements concernant les styles d'encodage et la translation des types natifs de données dans XML et vice versa.
- Encodage, décodage et routage des messages SOAP à travers le protocole de transport.

La couche de Communication SOAP (Simple Object Access Protocol)

STRUCTURE DES MESSAGES SOAP

La structure d'un message SOAP diffère selon les types d'informations transportées : Si ces informations sont tous textuelles, le format de message simple suffit, par contre s'il y a des informations nécessitant des formats de données binaires spécifique (image, vidéo, son, pdf,..) le format simple en suffit plus, et l'ensemble de ces informations doivent être encapsulées dans un format MIME qui sera indispensable.

Version SOAP		Valeur de l'URI utilisé
1.1	Enveloppe	http://schemas.xmlsoap.org/soap/envelope/
	Encodage	http://schemas.xmlsoap.org/soap/encoding/
1.2	Enveloppe	http://www.w3.org/2003/05/soap-envelope/
	Encodage	http://www.w3.org/2003/05/soap-encoding/

La couche de Communication SOAP (Simple Object Access Protocol)

1. Message simple :

Un message SOAP simple est message contenant uniquement du texte (XML). Il est composé de :

a) L'enveloppe SOAP (*Envelope*) :

C'est l'élément récipient (conteneur) qui englobe tous les autres éléments du message. Sa déclaration est strictement obligatoire puisque c'est elle qui détermine le début et la fin de la partie SOAP dans une requête. Le mot-clé ***Envelope*** (qui doit suivre le nom local de l'enveloppe) est suivi d'un espace de nom contenant un URI qui pointe vers un schéma XML. La valeur de cet URI (schéma XML) varie selon la version du message SOAP.

La couche de Communication SOAP (Simple Object Access Protocol)

Exemple : Déclaration de l'enveloppe d'un message SOAP écrit dans la version 1.1 et contenant un attribut ***encodingStyle*** :

```
<SOAP-ENV:Envelope
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
    ...
</SOAP-ENV:Envelope>
```

Un message SOAP peut commencer par une déclaration XML.

Exemple : Déclaration de l'enveloppe d'un message SOAP écrit dans la version 1.2 précédée d'une ligne XML (qui indique la version du langage XML utilisée) :

```
<?xml version="1.0"?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
    ...
</env:Envelope>
```

La couche de Communication SOAP (Simple Object Access Protocol)

b) L'entête du message (*Header*) :

C'est une partie optionnelle (souvent) qui sert comme mécanisme d'extension du protocole SOAP. Elle est composée d'une ou plusieurs entrées (Header entry/ Header Block), chacune d'elles peut être destinée à un ou plusieurs nœuds SOAP (nœuds intermédiaires et/ou récepteur final du message).

Chaque entrée peut contenir des attributs, parmi ces attributs il y en a quatre qui ont une importance dans le traitement du message: *encodingStyle, role (actor), mustUnderstand et relay :*

La couche de Communication SOAP (Simple Object Access Protocol)

- **Attribut « encodingStyle » :** Style d'encodage.
- **Attribut « *role* »** (***actor*** pour la version 1.1) **:** désigne le destinataire concerné par le traitement de cette entrée. Un destinataire peut être: un émetteur, un récepteur final ou bien aucun ou plusieurs intermédiaires.

  Exemple :

```
<SOAP-ENV:Header>
  <m:local xmlns:m =''http://www.w3edfRetD.edu/local/''
     soap:actor= http://www.w3edfRetD.edu/appli>
     <m:language> dk </m:language>
  </m:local>
</soap:Header>
```

La couche de Communication SOAP (Simple Object Access Protocol)

Cet attribut peut prendre 3 valeurs particulières:

Valeur	Nom	Description
next	"http://www.w3.org/2003/05/soap-envelope/role/next"	Tous les nœuds intermédiaires y compris le récepteur final
none	"http://www.w3.org/2003/05/soap-envelope/role/none"	Aucun nœud n'est concerné
UltimateReceiver	"http://www.w3.org/2003/05/soap-envelope/role/ultimateReceiver"	Récepteur final uniquement

La couche de Communication SOAP (Simple Object Access Protocol)

Dans le cas d'absence de cet attribut, tous les nœuds sont concernés.

Exemple :

```
<env:Header>
    <p:oneBlock                                    xmlns:p="http://example.com"
    env:role="http://example.com/Log"  env:mustUnderstand="true">
            ...
    </p:oneBlock>
    <q:anotherBlock                                xmlns:q="http://example.com"
    env:role="http://www.w3.org/2003/05/soap-envelope/ role/next">
            ...
    </q:anotherBlock>
    <r:aThirdBlock xmlns:r="http://example.com"> ...
    </r:aThirdBlock>
</env:Header>
```

La couche de Communication SOAP (Simple Object Access Protocol)

- **L'attribut « mustUnderstand » :** détermine si la compréhension de l'entrée par le nœud concerné est obligatoire ou non.

Exemple :

```
<soap:Header>
    <ns1:preferences soap:mustUnderstand='1'>
        <genre>Noir</genre>
        <format>WMV</format>
    </ns1:preferences>
    <ns2:priority>
        <level>low</level>
    </ns2:priority>
</soap:Header>
```

Dans cet exemple l'entrée « preferences » est obligatoire (attribut ***soap:mustUnderstand*** à "1."). Le service traitant ce message doit reconnaître ce header et le traiter correctement. Si ce header ne peut être traité correctement parce que le récepteur n'en a pas la capacité, alors un message d'erreur SOAP (***SOAP fault message***) doit être renvoyé.

La couche de Communication SOAP (Simple Object Access Protocol)

Exemple :

```
<soap:Header>
    <m:User xmlns:m="http://www.exemple.com/rights/"
      soap:actor="http://www.exemple.com/rights/RightsManager">
        Toto
    </m:User>
    <m:Session xmlns:m="http://www.exemple.com/session/"
      soap:mustUnderstand="1">12AE3C
    </m:Session>
    <m:Language xmlns:m="http://www.exemple.com/lang/"
        soap:actor="http://schemas.xmlsoap.org/soap/next"
        soap:mustUnderstand="0"> FR
    </m:Language>
</soap:Header>
```

Elément USER → destination = Nœud: *RightManager*
Elément Session → destination = ?

Elément Lang → destination = Nœud prochain.

La couche de Communication SOAP (Simple Object Access Protocol)

- **L'attribut « relay » :** Il détermine si une entrée dans l'entête d'un message SOAP arrivé chez un intermédiaire doit être réacheminée ou non lorsque le traitement de cette entrée est impossible par le nœud actuel. Cet attribut ne prend effet que sur les entrées non obligatoires destinées au nœud intermédiaire actuel.

Exemple :

```
<m:Language xmlns:m="http://www.example.com/lang/"
soap:role="http://www.w3.org/2003/05/soap-envelope/role/next"
soap:mustUnderstand="0"
soap:relay="true">
     FR
</m:Language>
```

La couche de Communication SOAP (Simple Object Access Protocol)

Résumé du comportement du réacheminement :

Valeur de l'attribut « role »		**Comportement du nœud**	
Valeur abrégée	**Nœud concerné par le traitement ?**	**Compris & traité ?**	**Réacheminé ?**
next	Oui	Oui	Non, sauf si réinséré
		Non	Non, sauf si relay ="true"
Défini par l'utilisateur	Oui	Oui	Non, sauf si réinséré
		Non	Non, sauf si relay ="true"
	Non	-	Oui
ultimateReceiver	Oui	Oui	-
		Non	-
	Non	-	Oui
none	Non	-	Oui

La couche de Communication SOAP (Simple Object Access Protocol)

c) Le corps du message (*Body*) :

C'est le bloc qui contient le corps du message. Il doit absolument être présent de manière unique dans chaque message et être contenu dans le bloc ***Envelope***. Les entrées de cette partie sont destinées toutes au récepteur final du message.

Chaque entrée peut contenir l'attribut ***encodingStyle***.

Le protocole SOAP n'exige l'utilisation d'aucune entrée prédéfinie dans le corps du message, sauf que pour le corps d'un message d'erreur qui ne doit contenir que l'entrée « ***fault*** » qui sert à reporter et expliquer la nature d'erreur rencontré.

La couche de Communication SOAP (Simple Object Access Protocol)

Exemple :

```
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header> ...
  </env:Header>
  <env:Body>
    <x:Book xmlns:x="http://www.examples.ws/">
      <x:Title>
      Service-Oriented Architecture
      A Field Guide to Integrating XML and Web services
      </x:Title>
    </x:Book>
  </env:Body>
</env:Envelope>
```

La couche de Communication SOAP (Simple Object Access Protocol)

2. Les messages d’erreur :

Les messages d’erreur sont des messages SOAP envoyés suite à une erreur dans le traitement d’un message de requête, ils servent à expliquer le type d’erreur à l’expéditeur du message requête.

Le corps d’un message d’erreur est constitué d’une entrée unique appelée l’entrée "Fault". Cette entrée sert à définir le type du message d’erreur, et à donner plus de détails sur cette erreur.

Selon le type d’erreur signalée, l’entête du message peut exister ou non. S’il existe il doit contenir des entrées spécifiques définies par le protocole SOAP.

La couche de Communication SOAP (Simple Object Access Protocol)

a) Structure de l'entrée Fault:

Version 1.2		Version 1.1
Code	**Value**	**Faultcode**
	Subcode	
Reason		**Faultstring**
Node		---
Detail		Detail
Role		Faultactor

La couche de Communication SOAP (Simple Object Access Protocol)

Exemple :

```
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
<s:Body>
<s:Fault>
<faultcode xmlns="">s:Client</faultcode>
<faultstring xml:lang="fr-FR" xmlns="">
Une opération invalide s'est produite.
</faultstring>
</s:Fault>
</s:Body>
</s:Envelope>
```

La couche de Communication SOAP (Simple Object Access Protocol)

b) Codes d'erreurs:

Code d'erreur		Désignation
Version 1.2	**Version 1.1**	
VersionMismatch	VersionMismatch	Erreur dans l'enveloppe (espace de nom ou nom local)
MustUnderstand	MustUnderstand	Elément dans l'en-tête n'est pas compris ou ne peut être traité
DataEncodingUnknown	--	Encodage d'un élément dans l'entête ou dans le corps non supporté par le nœud traitant
Sender	Client	Message incorrecte (structure) ou bien ne contient pas assez d'informations (informations d'authentification ou de payement oublies)
Receiver	Server	Erreur dans le traitement par le nœud (exemple temps dépassé)

La couche de Communication SOAP (Simple Object Access Protocol)

Exemple :

```
<env:Body>
  <env:Fault>
    <env:Code>
      <env:Value>
        env:VersionMismatch
      </env:Value>
    </env:Code>
    <env:Reason>
      <env:Text xml:lang="en">
        Versions do not match
      </env:Text>
    </env:Reason>
  </env:Fault>
</env:Body>
```

La couche de Communication SOAP (Simple Object Access Protocol)

3. Message avec pièce jointe :

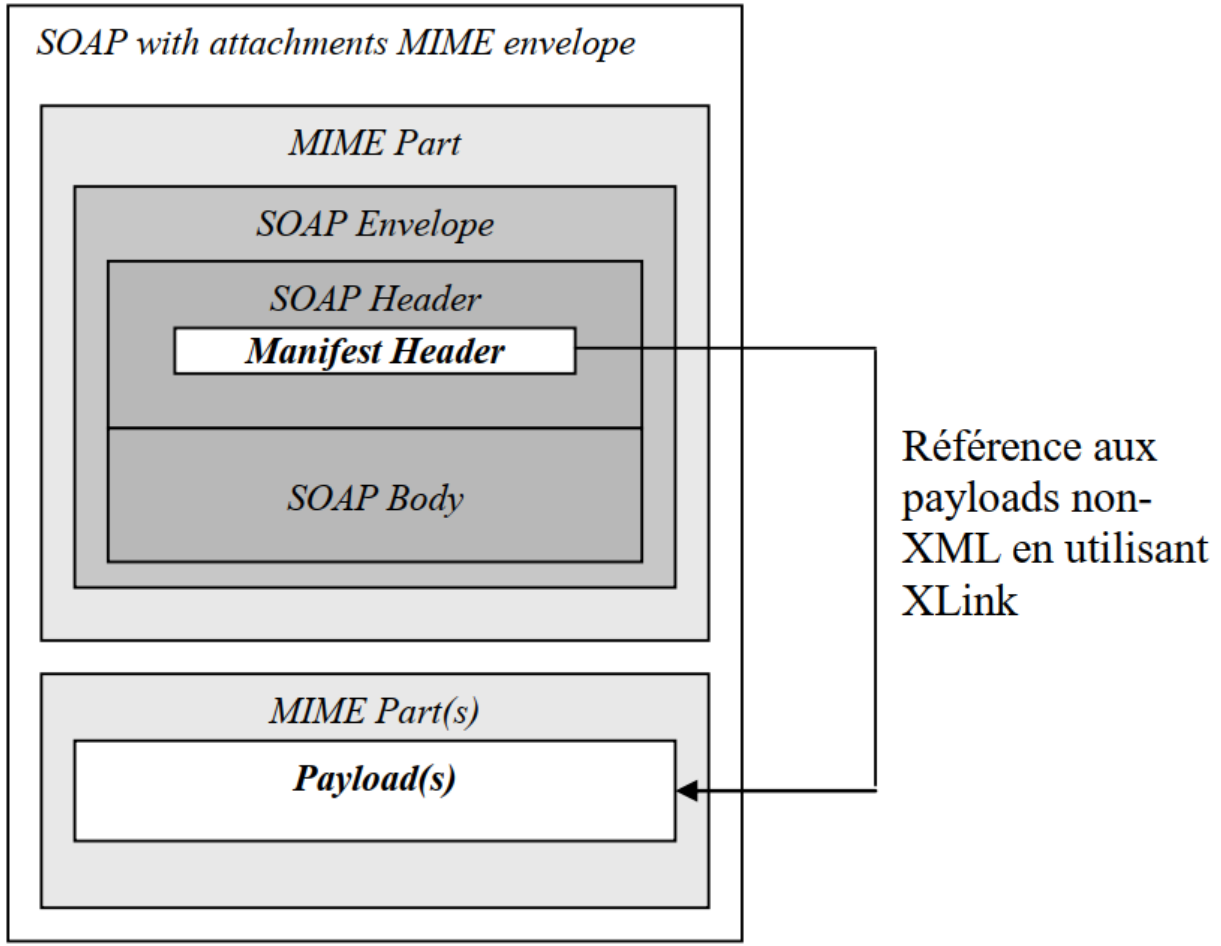
L'utilisation de SOAP avec des données non XML est parfois très utile, par exemple si on désire transmettre des images à partir d'une caméra à travers une liaison sans fil en utilisant SOAP vers un serveur client.

Le support des données non XML requiert un packaging complémentaire du message qui peut être fourni par une structure ***MIME multipart*** et impacte la liaison d'un message à son protocole de transport sous-jacent.

L'enveloppe MIME contient un ensemble de parties MIME individuelles, la 1ère contient le message SOAP qui doit comporter un bloc ***Manifest Header*** crée par le ***Handler Message Manifest***. La 2ème partie MIME contient les ***payloads*** qui peuvent être des documents XML ou d'autres formats.

La couche de Communication SOAP (Simple Object Access Protocol)

Un message avec attachement est structuré comme suit:



Structure d'un message avec attachements

La couche de Communication SOAP (Simple Object Access Protocol)

Exemple :

```
<env:Envelope xmlns:env='http://www.w3.org/2001/12/soap-envelope'
    <env:Header>
      <n:Manifest xmlns:n="http://exampleorg/manifest">
          <n:Reference n:id="image01"
                  Xlink:href="cid:payload-1"
                  Xlink:role="http://example.org/image">
             <n.description> Photo numero 1 </n:description>
          </n:Reference>
          <n:Reference n:id="image02"
             Xlink:href="cid:payload-2"
             Xlink:role="http://example.org/image">
             <n.description> Photo numero 2 </n:description>
          </n:Reference>
      </n:Manifest>
    </env:Header>
    <env:Body>
    . . .
    </env:Body>
</env:Envelope>
```

Le ***Manifest Header*** peut contenir les éléments qui référencent les parties MIME séparées en utilisant leurs identificateurs de contenu. Ceci peut être réalisé en utilisant les références XLink.

La couche de Communication SOAP (Simple Object Access Protocol)

SOAP en 2026 : Pourquoi il est toujours vivant?

- SOAP continue de rôder dans l'ombre des systèmes d'entreprise.

SOAP vs REST vs gRPC

- **SOAP :** Le protocole en « costume-cravate ». Strict, verbeux, mais très structuré.
- **REST :** Le « vendredi décontracté » des API. Flexible, orienté ressource, et fonctionne bien avec HTTP.
- **gRPC :** Le « petit nouveau ». Rapide, efficace, et idéal pour les microservices.

Alors que REST et gRPC ont volé la vedette, SOAP a tranquillement maintenu sa position dans certains secteurs. Dans certains domaines, il prospère.

La couche de Communication SOAP (Simple Object Access Protocol)

- L'une des forces de SOAP est sa compatibilité ascendante. Vous pouvez mettre à jour un service SOAP sans casser les clients existants, ce qui est une bonne nouvelle pour les entreprises avec de grands systèmes distribués qui ne peuvent pas tous être mis à jour simultanément.
- **SOAP dans les systèmes critiques :** Pour les systèmes où l'échec n'est pas une option, les normes complètes de SOAP pour la sécurité, la fiabilité et les transactions sont difficiles à battre. Pensez aux institutions financières, aux systèmes de santé et aux agences gouvernementales. SOAP est essentiel à leurs opérations quotidiennes.

La couche de Communication SOAP (Simple Object Access Protocol)

SOAP ne se limite pas à envelopper des appels de méthode dans du XML. Il est accompagné d'une suite de normes connues sous le nom de WS-* (Web Services) qui traitent divers aspects de la communication au niveau de l'entreprise.

- **Normes WS-* : Le couteau suisse des services Web**
- **WS-Security :** Fournit une sécurité de bout en bout pour les messages.
- **WS-ReliableMessaging :** Assure la livraison des messages même dans des conditions de réseau peu fiables.
- **WS-AtomicTransaction :** Prend en charge les transactions distribuées entre plusieurs services.
- **WS-Addressing :** Permet le routage des messages entre services indépendamment des protocoles de transport.

Ces normes font de SOAP une puissance pour les systèmes complexes et distribués où la sécurité et la fiabilité sont primordiales.

La couche de Communication SOAP (Simple Object Access Protocol)

SOAP dans le monde des microservices

- Bien que les microservices préfèrent généralement des protocoles plus légers comme REST ou gRPC, SOAP peut encore jouer un rôle :
- Intégration de systèmes hérités : Les services SOAP peuvent servir de pont entre les nouveaux microservices et les systèmes hérités.
- Opérations complexes : Pour les opérations nécessitant des transactions entre plusieurs services, le WS-AtomicTransaction de SOAP peut être inestimable.

La couche de Communication SOAP (Simple Object Access Protocol)

SOAP dans le cloud : Pas seulement un rêve

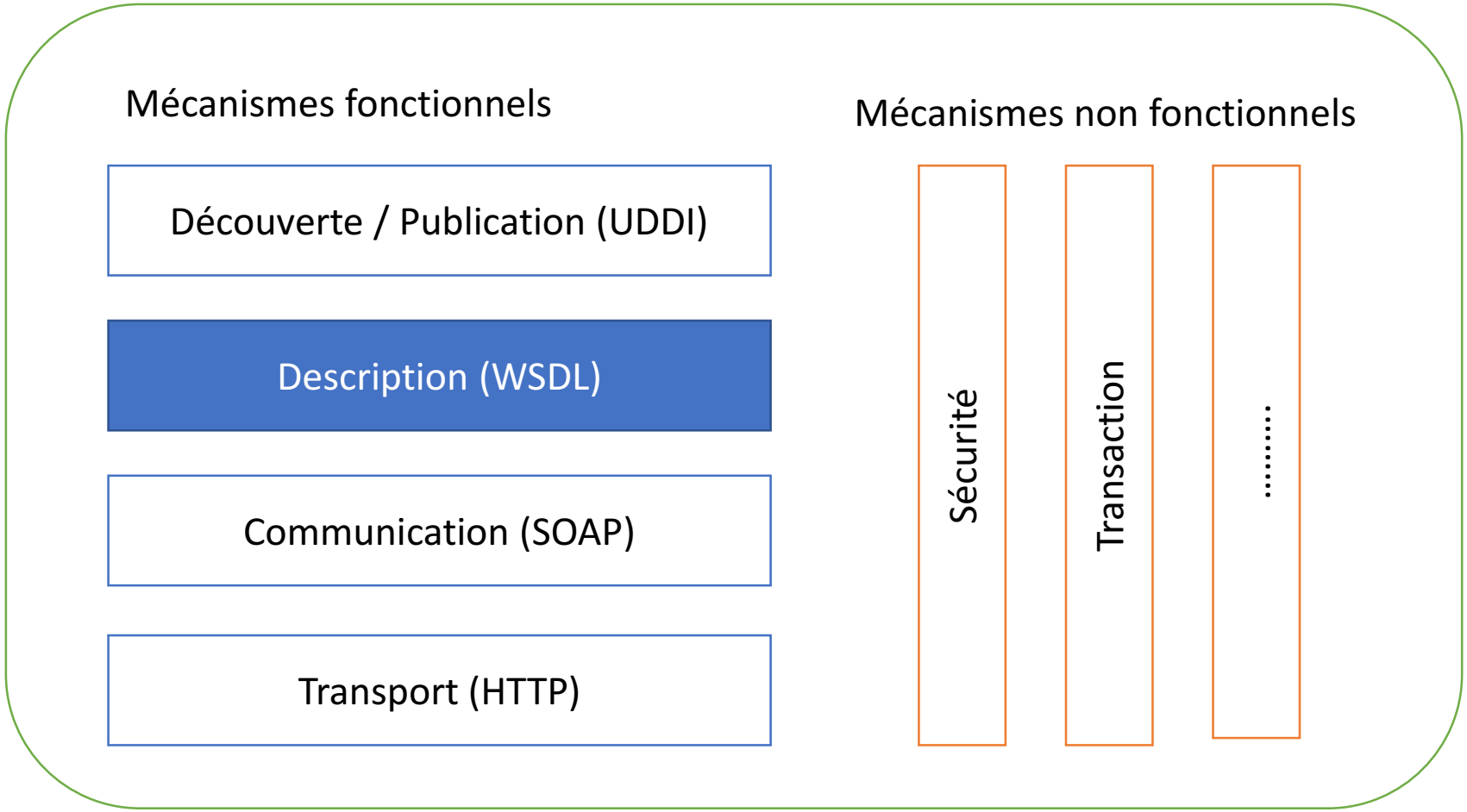
- Les principaux fournisseurs de cloud n'ont pas oublié SOAP :
- Amazon Web Services (AWS) prend en charge SOAP pour plusieurs de ses services, y compris Amazon EC2 et Amazon S3 (bien qu'ils recommandent REST pour les nouvelles implémentations).
- Microsoft Azure fournit des outils pour créer et consommer des services SOAP, reconnaissant son utilisation continue dans les environnements d'entreprise.

La couche de Communication SOAP (Simple Object Access Protocol)

Bibliothèques et frameworks modernes

- **Java :** Spring Web Services offre des outils puissants pour créer et consommer des services SOAP.
- **Python :** Zeep est un client SOAP moderne et convivial.
- **JavaScript :** Strong-soap est un client et serveur SOAP pour Node.js.
- **.NET :** WCF (Windows Communication Foundation) est toujours actif, et .NET Core a introduit de nouvelles façons de travailler avec SOAP.

La pile protocolaire simplifiée SOAP WS



La couche de Description WSDL (Web Service Description Language)

- Le protocole SOAP met à la disposition des services web un moyen standard de structuration et d'échange de messages XML. WSDL est un langage de description des capacités de services web basé sur XML. Un document WSDL décrit essentiellement le nom de la méthode utilisé, son nombre de paramètres, et leur type, ce qu'un service web offre, où il réside et comment on peut l'invoquer.
- La description d'un service doit inclure la définition des composants nécessaire au protocole de communication (SOAP pour les services web) et à l'interaction avec un client ou un autre service web.

La couche de Description WSDL (Web Service Description Language)

Les différents éléments décrits dans WSDL sont les suivants :

- Les **opérations** proposées par le service web.
- Les **données** et **messages** échangés lors de l'appel d'une opération.
- Le **protocole de communication.**
- Les **ports d'accès** au service.

Dans WSDL, il existe une séparation entre **deux niveaux indépendants, respectivement nommés abstrait et concret**. Le niveau abstrait regroupe les informations pouvant être réutilisées (non spécifique à un service), tandis que le niveau concret est constitué de la description des protocoles d'accès au service web (information particulière à un service). Le niveau abstrait est utilisé principalement lors du processus de sélection, tandis que le niveau concret est seulement utilisé lors de l'invocation des méthodes du service web.

La couche de Description WSDL (Web Service Description Language)

1. Niveau abstrait:

a) **Les types de données (*types*):** Le document WSDL permet de décrire les types de données échangées. WSDL supporte les types élémentaires (tels que les entiers, les chaînes de caractères) et les types complexes. Si les données échangées possèdent une structure particulière (*i.e.* un type complexe), il est possible de les décrire par l'intermédiaire d'un schéma XML.

b) **Les messages (*message*):** Un message correspond aux données qui sont véhiculées selon les méthodes invoquées. Chaque opération du service possède deux définitions de message : la première correspond à la requête et la seconde correspond à la réponse. La description d'un message contient le nom de l'élément en paramètre – d'entrée ou de sortie selon le type du message – et son type.

c) **Les opérations (*portType*)**: Une opération représente une unité de travail, c'est-à-dire une méthode proposée par le service web décrit. Chaque opération est identifiée par son nom.

La couche de Description WSDL (Web Service Description Language)

2. Niveau concret:

a) **Le protocole de communication (*binding*)**: La description des protocoles de communication permet de définir le protocole à utiliser pour l'appel des méthodes du service. Si nécessaire, le document WSDL peut contenir autant de descriptions de protocole que d'opérations, étant donné que le protocole de communication peut être différent pour chaque opération du service décrit.

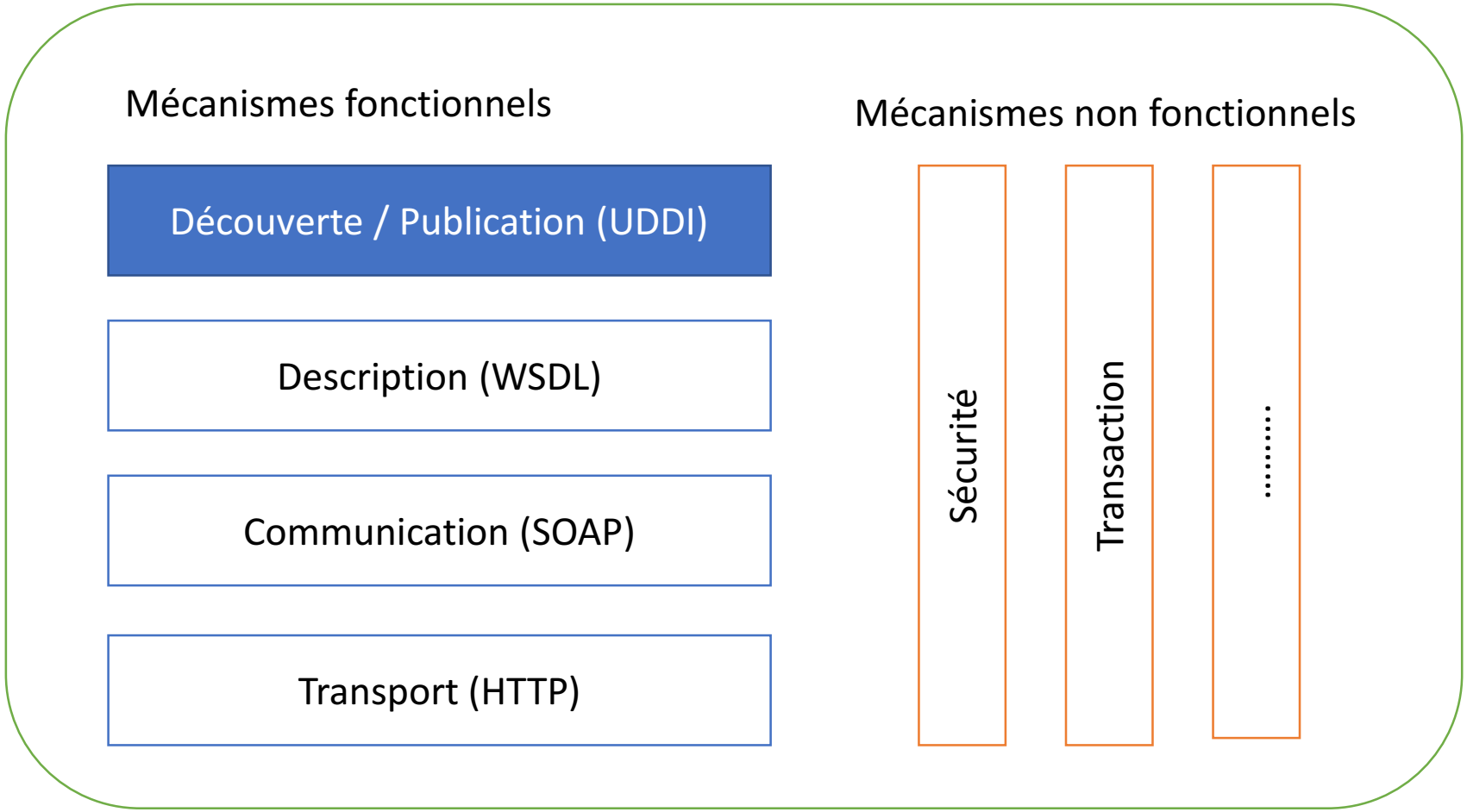
b) **Les ports d'accès au service (*service*)**: Dans un document WSDL, l'accès au service est défini par une collection de ports d'accès. Chaque port représente la localisation du service (i.e. son URL). Un même service peut être accessible sur plusieurs ports différents. Les informations contenues dans WSDL constituent la description du profil fonctionnel du service. Le client peut invoquer le service par le biais de sa description abstraite (méthodes disponibles, paramètres d'entrée et sortie) et concrète (description des protocoles de communication et des points d'accès du service).

La couche de Description WSDL (Web Service Description Language)

STRUCTURE D'UN DOCUMENT WSDL (Voir TP2)

	Concepts	WSDL 1.1	WSDL 2.0
Niveau abstrait	Types de données	<types/>	<types/>
	Opérations	<portType/>	<interface> <operation/> </interface>
	Messages	<message/>	<input/> <output/>
Niveau concret	Protocole de communication	<binding/>	<binding/>
	Ports d'accès	<service/>	<service/>

La pile protocolaire simplifiée SOAP WS



UDDI Moderne : Les Registres d'Entreprise Privés

- Le concept et la technologie de l'UDDI sont toujours vivants et pertinents aujourd'hui, mais sous une forme différente : en tant que registre de services privé au sein des grandes entreprises ou d'écosystèmes spécifiques.
- Les grandes organisations, notamment dans la finance, la gouvernement et les télécommunications, utilisent des registres UDDI privés pour gérer leurs écosystèmes internes de SOA (Architecture Orientée Services) ou de microservices.

UDDI Moderne : Les Registres d'Entreprise Privés

Scénario Réel : Une Grande Banque

- **Problème :** La banque dispose de centaines de services internes pour les données clients, les vérifications de crédit, le traitement des transactions et la détection de la fraude. Différents départements et applications doivent découvrir et utiliser ces services de manière fiable.
- **Solution :** Un registre UDDI privé.

UDDI Moderne : Les Registres d'Entreprise Privés

Comment cela fonctionne :

- Une équipe de développement crée un nouveau service SOAP/WSDL "Vérification d'Éligibilité aux Prêts".

Elle publie le fichier WSDL du service dans le **registre UDDI privé**. L'entrée du registre comprend :

- **Business Entity (Entité Commerciale) :** "Division des Prêts aux Particuliers"
- **Business Service (Service Commercial) :** "Service d'Éligibilité aux Prêts"
- **Binding Template :** L'URL technique (point de terminaison) où le service peut être appelé.
- **tModel :** Une "empreinte digitale" technique décrivant l'interface du service et les normes qu'il respecte (par ex., une version spécifique du protocole de sécurité interne).

UDDI Moderne : Les Registres d'Entreprise Privés

- Une autre équipe, qui construit une nouvelle application bancaire mobile, a besoin de vérifier l'éligibilité aux prêts. Elle interroge le registre UDDI pour trouver le "Service d'Éligibilité aux Prêts" officiel et approuvé, ainsi que son point de terminaison actuel.
- Le registre fournit le WSDL et le point de terminaison corrects, garantissant que l'application mobile s'intègre au bon service.

UDDI Moderne : Les Registres d'Entreprise Privés

Exemple Technologique Spécifique : Apache jUDDI

- C'est un registre UDDI v3 open-source entièrement fonctionnel, qui est toujours activement développé et utilisé.
- Les entreprises déploient jUDDI sur leurs propres serveurs pour créer un registre de services privé.
- Il peut être intégré à des bus de services comme Apache Camel et des plateformes de gouvernance.

Chapitre 3 : Web Services REST

Architecture des Web Services REST

- Acronyme de **RE**presentational **S**tate **T**ransfert défini dans la thèse de Roy Fielding en 2000.
- REST n'est pas un protocole ou un format, contrairement à SOAP, HTTP ou RCP, mais un style d'architecture inspiré de l'architecture du web fortement basé sur le protocole HTTP (peut utiliser d'autre protocoles).
- Architecture des Web Services REST est utilisée dans le développement des applications orientés ressources (ROA) ou orientées données (DOA).
- Les applications respectant l'architecture REST sont dites RESTful

Architecture des Web Services REST

Ce qu'il est :

- Un système d'architecture
- Une approche pour construire une application

Ce qu'il n'est pas

- Un protocole
- Un format
- Un standard

Architecture des Web Services REST

REST est une Architecture Simple basée sur 6 propriétés principales :

- Performance : Interaction simple entre les composants
- Evolutivité : Supporte une large variété de composants
- Simplicité : Entre les interfaces
- Modification : Peut être modifié sans impacter les clients
- Visibilité : Communication claire entre les composantes
- Confiance : Reprise sur panne

Architecture des Web Services REST

Caractéristiques :

- Séparation Client/Serveur
- Sans Etats (Stateless) : Chaque requête envoyée au serveur doit contenir toutes les informations relatives à son état et est traitée indépendamment de toutes autres requêtes / Aucune gestion de session ou d'état des ressources par le serveur
- Possibilité de mise en Cache
- Orienté Ressources
- Messages auto-descriptifs
- Hypermedia : Aucune autre action n'est supportée outre celles qui sont décrites

Architecture des Web Services REST

- Une ressource est un objet identifiable sur le système (Livre, Client, Prêt, ..)
- Une ressources n'est pas forcément une entité physique, elle peut être virtuelle (Prêt, Consultation,…)
- Une ressource est identifiée par une URI : Une URI identifie uniquement une ressource sur le système
- Une ressource peut subir des actions correspondant aux opérations effectuées sur la ressource.
- Actions sur les ressources : Les opérations de bases (CRUD) sont identifiées par les types de requêtes HTTP (GET, PUT, POST, DELETE)

Architecture des Web Services REST

Une représentation désigne les données échangées entre le client et le serveur pour une ressource:

- GET : Le serveur renvoie au client l'état de la ressource
- PUT, POST : Le client envoie l'état d'une ressource au serveur. Peut être sous différent format : JSON, XML, XHTML, CSV, Text/plain, ...
- DELETE :

Exemple:

- GET /api/students → Récupérer tous les étudiants
- GET /api/students/1 → Récupérer l'étudiant n°1
- POST /api/students → Créer un nouvel étudiant
- PUT /api/students/1 → Modifier l'étudiant n°1
- DELETE /api/students/1 → Supprimer l'étudiant n°1

Architecture des Web Services REST

JSON « **J**ava**S**cript **O**bject **N**otation » est un format d'échange de données, facile à lire par un humain et interpréter par une machine.

Basé sur JavaScript, il est complètement indépendant des langages de programmation mais utilise des conventions qui sont communes à toutes les langages de programmation (C, C++, Perl, Python, Java, C#, VB, JavaScript,….)

Deux structures :

- Une collection de clefs/valeurs : Object
- Une collection ordonnée d'objets : Array

Architecture des Web Services REST

WADL : Web Application Description Language

- Langage de description des services REST au format XML / Standard du W3C
- Permet de décrire les éléments des services : Resource, Méthode, Paramètre, Réponse
- Permet d'interagir de manière dynamique avec les applications REST
- Fournit les informations descriptives d'un service permettant de construire des applications clientes exploitant les services REST.
- Moins exploité que le WSDL pour les Services SOAP

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<application xmlns="http://wadl.dev.java.net/2009/02">
   <doc xmlns:jersey="http://jersey.java.net/" jersey:generatedBy="Jersey: 2.0 2013-05-03 14:50:15"/>
   <grammars/>
  ▼<resources base="http://localhost:8080/Bibliotheque/webresources/">
    ▼<resource path="category">
      ▼<method id="test" name="GET">
        ▼<response>
            <representation mediaType="application/xml"/>
            <representation mediaType="application/json"/>
          </response>
        </method>
      ▼<method id="apply" name="OPTIONS">
        ▼<request>
            <representation mediaType="*/*"/>
          </request>
        ▼<response>
            <representation mediaType="application/vnd.sun.wadl+xml"/>
          </response>
        </method>
      ▼<method id="apply" name="OPTIONS">
        ▼<request>
            <representation mediaType="*/*"/>
          </request>
        ▼<response>
            <representation mediaType="text/plain"/>
          </response>
        </method>
      ▼<method id="apply" name="OPTIONS">
        ▼<request>
            <representation mediaType="*/*"/>
          </request>
        ▼<response>
            <representation mediaType="*/*"/>
          </response>
        </method>
```

```
▼<resource path="application.wadl">
  ▼<method id="getWadl" name="GET">
    ▼<response>
        <representation mediaType="application/vnd.sun.wadl+xml"/>
        <representation mediaType="application/xml"/>
      </response>
    </method>
  ▼<method id="apply" name="OPTIONS">
    ▼<request>
        <representation mediaType="*/*"/>
      </request>
    ▼<response>
        <representation mediaType="text/plain"/>
      </response>
    </method>
  ▼<method id="apply" name="OPTIONS">
    ▼<request>
        <representation mediaType="*/*"/>
      </request>
    ▼<response>
        <representation mediaType="*/*"/>
      </response>
    </method>
```

Architecture des Web Services REST

Implémentation d'un web service REST - JAX-RS API

- Développer des Web Services REST avec JAVA
- JAX-RS : Acronyme de Java API for RestFul Web Services

Version courante 2.0 décrite par JSR 339

Depuis la version 1.1, il fait partie intégrante de la spécification Java EE 6. Il décrit la mise en œuvre des services REST web coté serveur. Son architecture se repose sur l'utilisation des classes et des annotations pour développer les services web.

Architecture des Web Services REST

JAX-RS est une spécification et autour de cette spécification sont développés plusieurs implémentations

- JERSEY : implémentation de référence fournie par Oracle (http://jersey.java.net) Version actuelle 2.24.1 implémentant les spécifications de JAX-RS 2.0 / Intégré dans Glassfish et l'implémentation Java EE (6,7) / Supportés dans Netbeans
- CXF : Fournie par Apache (http://cfx.apache.org)
- RESTEasy : fournie par JBOSS
- RESTLET : L'un des premiers framework implémentant REST pour Java

Architecture des Web Services REST

- JAX-RS : Basé sur POJO (Plain Old Java Object) en utilisant des annotations spécifiques JAX-RS
- Pas de modifications dans les fichiers de configuration
- Le service est déployé dans une application web
- Pas de possibilité de développer le service à partir d'un WADL contrairement à SOAP
- Approche Bottom/Up : Développer et annoter les classes / Le WSDL est automatiquement généré par l'API

Architecture des Web Services REST

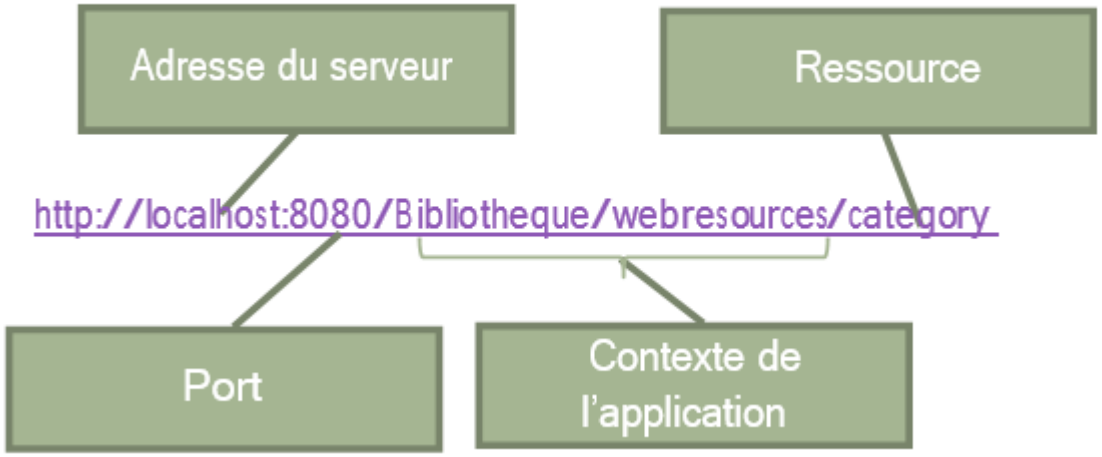
La spécification JAX-RS dispose d'un ensemble d'annotation permettant d'exposer une classe et ses méthodes dans un services web :

- @Path
- @GET, @POST, @PUT, @DELETE
- @Produces, @Consumes
- @PathParam

Architecture des Web Services REST

- L'annotation @Path permet de définir l'URI des ressources modélisés par une classe A positionner au-dessus de la déclaration d'une classe
- Syntaxe : @Path("nom_de_la_ressource")
- Expose la classe comme ressource dans le WS
- @Path : Définit la racine des ressources (Root Racine Ressources)
- Sa valeur correspond à l'URI relative de la ressource

```
@Path("category")
public class CategoryService {
.....
}
```



Architecture des Web Services REST

```
@Path("hello")
public class HelloWorld{ @GET
@Produces(MediaType.TEXT_PLAIN) public String sayHello()
{
return "Hello World!";
}
}
```

Architecture des Web Services REST

@Path peut être utilisée pour annoter des méthodes d'une classe. Permet de définir des URIs pour les actions sur une ressource. L'URI résultante est la concaténation entre la valeur de @Path de la classe et celle de la méthode

```
 @Path( "hello")
public class CategoryFacade {
@GET
@Produces(MediaType.TEXT_PLAIN) @Path("greetings")
public String hello()
{
return "Hello World!";
}
}
```

Architecture des Web Services REST

@Produces placée sur les méthodes. Spécifie le format de données renvoyées par la méthode. Correspond à la clef Content-Type de l'en tête des requêtes HTTP.

@Consumes spécifie le format des données d'entrées de la méthode. Placée au dessus de la déclaration des méthodes. Correspond à la clef Accept de l'en tete des requetés HTTP.

Architecture des Web Services REST

Valeurs @Consumes/@Produces

Plusieurs Mime-Type

- text/plain
- application/json
- application/xml
- application/x-www-form-urlencoded
- Application/csv
- …

Architecture des Web Services REST

Annotation dynamique

- La valeur définie dans l'annotation @Path n'est forcément une constante, elle peut être variable.
- Possibilité de définir des expressions plus complexes, appelées Template Parameters
- Les contenus complexes sont délimités par « {} »
- Possibilité de mixer dans la valeur @Path des expressions régulières

Paramètres dynamiques

- @PathParam : Récupérer dans l'URL (Path)
- @FormParam : Récupérer dans le formulaire (Form)
- @QueryParam : Récupérer dans les paramètres de l'URL (Query)
- @HeaderParam : Récupérer dans l'entête de la requête (Header)
- @CookieParam : Récupérer les informations de cookies

Architecture des Web Services REST

Exemple : Dire bonjour à la personne passée en paramètre

```
@Path( "hello")
public class HelloWorld{ @GET
@Produces(MediaType.TEXT_PLAIN) @Path("calcul")
public String calculPost(@FormParam("nom") String nom){
return "Hello " + nom;
}
}
```

Architecture des Web Services REST

Exemple : Calcule la somme de deux nombres passés en paramètre

```
@Path( "hello")
public class HelloWorld{ @GET
@Produces(MediaType.TEXT_PLAIN) @Path( "calcul/{nb1}+{nb2}")
public double calcul(@PathParam( "nb1") nb1),
(@PathParam( "nb2") int nb2){ return nb1 + nb2;
}
}
```

Architecture des Web Services REST

Exemple : Calcule la somme de deux nombres passés en paramètre

```
@Path( "hello")
public class HelloWorld{ @GET
@Produces(MediaType.TEXT_PLAIN) @Path( "calcul/ ")
public double calcul(@QueryParam( "nb1") nb1, (@ QueryParam( "nb2") int nb2){
return nb1 + nb2;
}
}
```

Architecture des Web Services REST

Quatre types principaux:

- @GET : Opération de lecture (Read)
- @POST : Opération d’écriture (Create)
- @PUT : Opération d’écriture (Create/Update)
- @DELETE : Opération de suppression (Delete)

Architecture des Web Services REST

@GET Placé sur les méthodes à exposer

Pour des opérations de lecture

Répond uniquement aux requêtes HTTP de type GET

Accepte des données dans l'URL de la requête

Une annotation @Consumes serait obsolète

@Produces format des données de retour (Obligatoire)

Architecture des Web Services REST

@POST pour annoter les méthodes

Précise que la méthode n'est accessible que par des requêtes HTTP de type POST

Opération d'écriture (Create)

Doit recevoir logiquement des paramètres dans le corps de la requête

Peut accepter des paramètres dans l'URL

@Consumes pour définir le format des données d'entrée

@Produces format des données de retour (optionnel)

Architecture des Web Services REST

@PUT Précise que la méthode n’est accessible que par des requêtes HTTP de type POST

Opération d’écriture (Create, Update)

Doit recevoir logiquement des données dans le corps de la requête

Peut accepter des paramètres dans l’URL

@Consumes pour définir le format des données d’entrée

@Produces format des données de retour (optionnel)

Architecture des Web Services REST

@DELETE Précise que la méthode n'est accessible que par des requêtes HTTP de type DELETE

Opération de suppression (Delete)

Peut accepter des paramètres dans l'URL

@Consumes serait obsolète

@Produces format des données de retour (pas obligatoire)

Architecture des Web Services REST

@FormParam Permet de recevoir des données d'un formulaire
Associé aux données d'entrée de type application/x- www-form-urlencoded

```
@Path( "hello")
public class HelloWorld{ @GET
@Produces(MediaType.TEXT_PLAIN)
@Consumes(MediaType. APPLICATION_FORM_URLENCODED)
@Path( "calcul/ ")
public double calcul(@FormParam( "nb1") nb1,
(@ FormParam( "nb2") int nb2)
{return nb1 + nb2;}}
```

Architecture des Web Services REST

- @get @post @put @delete Permettent de mapper une méthode à un type de requête HTTP
- Ne sont utilisables que sur des méthodes
- Plusieurs méthodes peuvent avoir le même chemin, le mapping uri/méthode est fait automatiquement par JAX-RS en fonction du type de la requête

Architecture des Web Services REST

- La spécification JAXB permet d'utiliser des types personnalisés (i.e. Class)
- JAXB permet de faire le mapping XML à POJO
- Manipuler les objets directement sans avoir à gérer directement du XML
- JAXB définit des annotations permettant de faire le mapping : @XmlRootElement, @XmlElement, @XmlType

Architecture des Web Services REST

Toute class (entité) annoté par @XmlRootElement est automatiquement mappé en XML

Le format JSON est également supporté

Le format adopté dépend des annotations

- @Consumes à Désérialisation
- @Produces à Sérialisation

XML : text/xml, application/xml…

JSON : application/json

Plateformes de Gestion des API

- Pour les API REST modernes et les microservices, l'UDDI a été largement remplacé par **les Plateformes de Gestion des API**.
- Ces plateformes offrent les mêmes fonctions principales, description, découverte et intégration, mais d'une manière plus conviviale et centrée sur le web.

Plateformes de Gestion des API

Concept UDDI	Équivalent Moderne (Gestion des API)
Business Entity	Fournisseur d'API / Équipe
Business Service	Produit API / Collection
Binding Template	Route / Endpoint de la Passerelle API
tModel	Schéma d'API (OpenAPI/Swagger)
Registre UDDI	Portail Développeur / Catalogue d'API

Exemples

- **Google Cloud API Gateway / Apigee :**
  - Vous publiez la **Spécification OpenAPI** de votre API (le WSDL moderne) sur Apigee.
  - Apigee agit comme la passerelle et le registre gérés.
  - Les développeurs découvrent les API disponibles via un **Portail Développeur**, qui est un catalogue de tous les services, consultable et documenté — remplissant directement le rôle de "Découverte" de l'UDDI.
- **AWS API Gateway & AWS Service Catalog :**
  - Vous déployez votre API sur AWS API Gateway.
  - Vous pouvez enregistrer votre API en tant que produit dans **AWS Service Catalog** pour la découverte interne et la gouvernance.
  - Les équipes de l'organisation peuvent parcourir le Service Catalog pour trouver et consommer les API internes approuvées.

Exemples

- **Azure API Management :**
  - Vous importez le schéma OpenAPI de votre API dans Azure API Management.
  - Il génère automatiquement un **portail développeur** qui agit comme un registre de services. Les autres développeurs de votre organisation peuvent visiter ce portail pour découvrir, apprendre et obtenir les clés pour utiliser votre API.
- **Open-Source : Kong Gateway & Kong Developer Portal :**
  - Similaire aux fournisseurs cloud, Kong fournit une passerelle et un portail développeur personnalisable où toutes les API enregistrées sont listées et documentées pour une découverte et une intégration faciles.

Exemple REST

- **Étape 1 : Configuration de la base de données**
- **Étape 2 : Ajout des dépendances pour MySQL et javax.persistence**
- **Étape 3 : Création des entités JPA**
- **Étape 4 : Création d'un Service**

Étape 1 : Configuration de la base de données (application.properties)

Si vous utilisez **H2** en base de données en mémoire, ajoutez les propriétés suivantes à src/main/resources/application.properties :

```
# Configuration pour H2 Database
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

Si vous utilisez **MySQL**, vous devez configurer le fichier comme suit :

```
# Configuration pour MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/nom_de_la_base
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
spring.datasource.username=root
spring.datasource.password=motdepasse
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
```

Étape 2 : Ajout des dépendances pour MySQL et javax.persistence (Fichier Pom.xml)

```
<dependency>
   <groupId>mysql</groupId>
   <artifactId>mysql-connector-java</artifactId>
</dependency>
<dependency>
        <groupId>javax.persistence</groupId>
<artifactId>javax.persistence-api</artifactId>  <version>2.2</version> <!-- ou la version qui vous convient -->
</dependency>
```

Étape 3 : Création des entités JPA
3.1 : Création d'une entité Utilisateur

Dans le répertoire src/main/java/com/example/springbootdb/model, créez une classe Utilisateur :

```
package com.example.springbootdb.model;
 import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;


 @Entity
public class Utilisateur {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nom;
    private String email;
    // Getters et Setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

Étape 3 : Création des entités JPA
3.2 : Création d'un Repository

Dans le répertoire src/main/java/com/example/springbootdb/repository, créez une interface UtilisateurRepository qui étend JpaRepository :

```
package com.example.springbootdb.repository;


import com.example.springbootdb.model.Utilisateur;
import org.springframework.data.jpa.repository.JpaRepository;


public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
    // Nous pouvons ajouter des méthodes de recherche personnalisées ici, par exemple :
    Utilisateur findByNom(String nom);
}
```

Étape 4 : Création d'un Service

Dans src/main/java/com/example/springbootdb/service, créez une classe UtilisateurService :

```
package com.example.springbootdb.service;

import com.example.springbootdb.model.Utilisateur;
import com.example.springbootdb.repository.UtilisateurRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class UtilisateurService {
    @Autowired
    private UtilisateurRepository utilisateurRepository;

    public List<Utilisateur> getAllUtilisateurs() {
        return utilisateurRepository.findAll();
    }

    public Optional<Utilisateur> getUtilisateurById(Long id) {
        return utilisateurRepository.findById(id);
    }

    public Utilisateur saveUtilisateur(Utilisateur utilisateur) {
        return utilisateurRepository.save(utilisateur);
    }

    public void deleteUtilisateur(Long id) {
        utilisateurRepository.deleteById(id);
    }
}
```

Étape 5 : Création d'un contrôleur REST

Dans src/main/java/com/example/springbootdb/controller, créez un contrôleur UtilisateurController :

```
package com.example.springbootdb.controller;

import com.example.springbootdb.model.Utilisateur;

import com.example.springbootdb.service.UtilisateurService;

import
org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;

import java.util.List;

import java.util.Optional;

 @RestController

@RequestMapping("/api/utilisateurs")

public class UtilisateurController {

    @Autowired

   private UtilisateurService utilisateurService;

    @GetMapping

    public List<Utilisateur> getAllUtilisateurs() {

        return utilisateurService.getAllUtilisateurs();

    }

     @GetMapping("/{id}")

    public Optional<Utilisateur> getUtilisateurById(@PathVariable
Long id) {

        return utilisateurService.getUtilisateurById(id);

    }

     @PostMapping

   public Utilisateur saveUtilisateur(@RequestBody Utilisateur
utilisateur) {

        return utilisateurService.saveUtilisateur(utilisateur);

    }

     @DeleteMapping("/{id}")

    public void deleteUtilisateur(@PathVariable Long id) {

        utilisateurService.deleteUtilisateur(id);  }}
```

Étape 6 : Lancer l'application

Exécutez la classe principale de l'application Spring Boot, par exemple SpringbootDbApplication.java.

```
package com.example.springbootdb;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
 @SpringBootApplication
public class SpringbootDbApplication {
   public static void main(String[] args) {
       SpringApplication.run(SpringbootDbApplication.class, args);
    }
}
```

L'application sera accessible sur http://localhost:8080/.

Étape 7 : Tester l'application

Vous pouvez maintenant tester votre API avec un outil comme **Postman** ou **curl** :
GET /api/utilisateurs : pour obtenir tous les utilisateurs.
GET /api/utilisateurs/{id} : pour obtenir un utilisateur par ID.
POST /api/utilisateurs : pour ajouter un utilisateur.
DELETE /api/utilisateurs/{id} : pour supprimer un utilisateur.
Exemples de requêtes :
GET http://localhost:8080/api/utilisateurs
POST http://localhost:8080/api/utilisateurs avec un body JSON comme :

```
{
   "nom": "Jean Dupont",
   "email": "jean.dupont@example.com"
}
```

Chapitre 4 : Web Services gRPC

gRPC (Remote Procedure Calls)

- gRPC (Remote Procedure Calls) est un framework RPC open-source développé par Google qui utilise **HTTP/2** comme protocole de transport et **Protocol Buffers** (protobuf) comme format d'échange de données et comme interface de définition de service (IDL).
- Idéal pour la communication inter-services
- Architecture gRPC

Client → Protobuf (sérialisation) → HTTP/2 → Serveur → Protobuf (désérialisation)

Workflow de Développement

- **Définition : Création du fichier .proto**
- **Génération : Compilation pour générer le code client/serveur**
- **Implémentation : Développement de la logique métier**
- **Déploiement : Lancement du serveur et connexion des clients**

Caractéristiques

- Performance : Format binaire (Protocol Buffers)
- Bidirectionnel : Communication dans les deux sens
- Multi-langage : Client et serveur dans différents langages
- Streaming : Envoi de données en continu

Avantages Principaux de gRPC

Performances Supérieures

- **Binaire** : Format Protobuf plus compact que JSON
- **HTTP/2** : Multiplexing, compression des headers, connexions persistantes
- **Faible latence** : Sérialisation/désérialisation rapide

Contrat Strict

- Interface bien définie via les fichiers .proto
- Génération de code automatique pour multiple langages
- Validation automatique des données

Avantages Principaux de gRPC

Modèles de Communication Avancés

- **Unary RPC** : Requête-Réponse classique
- **Server Streaming** : Le serveur envoie un flux de réponses
- **Client Streaming** : Le client envoie un flux de requêtes
- **Bidirectional Streaming** : Communication bidirectionnelle en temps réel

Mailles de Services (Service Meshes) : L'Évolution des Microservices

- Dans un monde cloud-native, basé sur Kubernetes, le rôle de la découverte de services est devenu encore plus automatisé et est géré par les **Mailles de Services (Service Meshes)**.
- **Exemple : Istio Service Mesh**
- **Comment cela fonctionne :** Vous ne "publiez" pas manuellement un service. Lorsque vous déployez un microservice sur Kubernetes, vous définissez une ressource **Kubernetes Service**. Un maillage de services comme Istio découvre automatiquement ce service.

Mailles de Services (Service Meshes) : L'Évolution des Microservices

- **Le "Registre" :** Le **plan de contrôle (control plane)** du maillage de services (Istiod) agit comme un registre de services dynamique en temps réel. Il connaît chaque instance de service (pod) et son état de santé.
- **Découverte & Intégration :** Le "plan de données (data plane)" (les proxies Envoy) interroge le plan de contrôle pour trouver les instances disponibles d'un service qu'ils doivent appeler. Ceci est entièrement automatisé et transparent pour le développeur.

Époque	Technologie	Cas d'Usage Principal	Exemple de "Registre"
Années 2000	UDDI Public	Découverte Globale des Services Web	(Défunt : nœuds publics IBM, Microsoft)
Ère SOA	UDDI Privé	Gouvernance SOA Interne	Apache jUDDI déployé sur site
Économie des API	Gestion des API	Gérer et Monétiser les API REST	Google Apigee, Azure API Management
Cloud-Native	Mailles de Services	Communication Automatisée des Microservices	Istio, Linkerd, Consul (plan de contrôle)

Exemple gRPC

Implémentation avec Spring Boot

- Étape 1 : Dépendances
- Étape 2 : Définition du service Protobuf
- Étape 3 : Implémentation du service gRPC (Fichier : StudentGrpcService.java)
- Étape 4 : Configuration gRPC (Fichier : application.properties)

Étape 1 : Dépendances (Fichier pom.xml)

```
<dependencies>
    <!-- gRPC Spring Boot Starter -->
    <dependency>
        <groupId>net.devh</groupId>
        <artifactId>grpc-server-spring-boot-starter</artifactId>
        <version>2.14.0.RELEASE</version>
    </dependency>
        <!-- Protobuf Java -->
    <dependency>
   <groupId>com.google.protobuf</groupId>
        <artifactId>protobuf-java</artifactId>
        <version>3.21.12</version>
    </dependency>
</dependencies>
```

```
 <build>
    <plugins>
        <!-- Plugin pour générer le code Protobuf -->
        <plugin>
<groupId>org.xolstice.maven.plugins</groupId>
            <artifactId>protobuf-maven-plugin</artifactId>
            <version>0.6.1</version>
            <configuration>
<protocArtifact>com.google.protobuf:protoc:3.21.12:exe:${os.detected.classifier}</protocArtifact>
                <pluginId>grpc-java</pluginId>
                <pluginArtifact>io.grpc:protoc-gen-grpc-java:1.52.1:exe:${os.detected.classifier}</pluginArtifact>
            </configuration>
            <executions>
                <execution>
                    <goals>
                        <goal>compile</goal>
                        <goal>compile-custom</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

Étape 2 : Définition du service Protobuf (Fichier : src/main/proto/student.proto)

```
syntax = "proto3";

 option java_package = "com.example.university.grpc";

option java_multiple_files = true;

 // Définition du service gRPC

service StudentService {

  // RPC simple : un request → un response

  rpc GetStudent (GetStudentRequest) returns (StudentResponse);

    // RPC pour créer un étudiant

  rpc CreateStudent (CreateStudentRequest) returns (StudentResponse);

    // Streaming : serveur envoie plusieurs réponses

  rpc GetAllStudents (GetAllStudentsRequest) returns (stream StudentResponse);

}

 // Messages (équivalents aux DTOs)

 message GetStudentRequest {

  int64 student_id = 1;

}

 message CreateStudentRequest {

  string first_name = 1;

  string last_name = 2;

  string email = 3;

  int32 age = 4;

}

 message GetAllStudentsRequest {

  // Peut être vide ou contenir des filtres

}

 message StudentResponse {

  int64 student_id = 1;

  string first_name = 2;

  string last_name = 3;

  string email = 4;

  int32 age = 5;

}
```

Étape 2 : Définition du service Protobuf (Fichier : src/main/proto/student.proto)

- service : Définit les méthodes RPC
- message : Structures de données (comme les classes)
- rpc : Définition d'une méthode distante
- stream : Permet l'envoi/réception de multiples messages

Étape 3 : Implémentation du service gRPC (Fichier : StudentGrpcService.java)

```
package com.example.university.grpc;
 import com.example.university.models.Student;
import com.example.university.services.StudentService;
import io.grpc.stub.StreamObserver;
import net.devh.boot.grpc.server.service.GrpcService;
import
org.springframework.beans.factory.annotation.Autowired;
 import java.util.List;
 @GrpcService
public class StudentGrpcService extends
StudentServiceGrpc.StudentServiceImplBase {
    @Autowired
   private StudentService studentService;
    @Override
   public void getStudent(GetStudentRequest request,
                          StreamObserver<StudentResponse>
responseObserver) {
       Long studentId = request.getStudentId();
       try {
          Student student =
studentService.getStudentById(studentId)
                  .orElseThrow(() -> new
RuntimeException("Étudiant non trouvé"));
```

```
// Convertir l'entité Student en StudentResponse gRPC
StudentResponse response = StudentResponse.newBuilder()
        .setStudentId(student.getId())
        .setFirstName(student.getFirstName())
        .setLastName(student.getLastName())
        .setEmail(student.getEmail())
        .setAge(student.getAge())
        .build();
        // Envoyer la réponse
        responseObserver.onNext(response);
        responseObserver.onCompleted();
        } catch (Exception e) {

responseObserver.onError(io.grpc.Status.NOT_FOUND
                .withDescription("Étudiant non trouvé")
                .asRuntimeException());
            }
        }
```

```
 @Override
public void createStudent(CreateStudentRequest request, StreamObserver<StudentResponse> responseObserver) {
try {
        // Convertir la request gRPC en entité Student
        Student student = new Student(
                request.getFirstName(),
                request.getLastName(),
                request.getEmail(),
                request.getAge()
        );
Student savedStudent = studentService.createStudent(student);
        // Construire la réponse
        StudentResponse response = StudentResponse.newBuilder()
                .setStudentId(savedStudent.getId())
                .setFirstName(savedStudent.getFirstName())
                .setLastName(savedStudent.getLastName())
                .setEmail(savedStudent.getEmail())
                .setAge(savedStudent.getAge())
                .build();
                   responseObserver.onNext(response);
            responseObserver.onCompleted();
                } catch (Exception e) {
responseObserver.onError(io.grpc.Status.ALREADY_EXISTS
                .withDescription("Erreur création étudiant: " + e.getMessage())
                .asRuntimeException());
       }
    }
```

```
@Override
public void getAllStudents(GetAllStudentsRequest request,
StreamObserver<StudentResponse> responseObserver) {
List<Student> students = studentService.getAllStudents();
// Envoyer chaque étudiant dans le stream
    for (Student student : students) {
        StudentResponse response = StudentResponse.newBuilder()
                .setStudentId(student.getId())
                .setFirstName(student.getFirstName())
                .setLastName(student.getLastName())
                .setEmail(student.getEmail())
                .setAge(student.getAge())
                .build();
        responseObserver.onNext(response);
    }
// Terminer le stream
    responseObserver.onCompleted();
  }
}
```

Étape 4 : Configuration gRPC (Fichier : application.properties)

```
# Port gRPC
grpc.server.port=9090

# Base de données H2 (développement)
spring.datasource.url=jdbc:h2:mem:universitydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Console H2
spring.h2.console.enabled=true
```

Bilan SOAP

Avantages

- Standardisé
- Interopérabilité
- Sécurité (WS-Security)

Inconvénients

- Performances (enveloppe SOAP supplémentaire)
- Complexité, lourdeur

Bilan REST

Avantages

- Simplicité de mise en œuvre
- Lisibilité par un humain
- Evolutivité
- Repose sur les principes du web
- Représentations multiples (XML, JSON,…)
- API publique
- Communication browser-frontend
- Développement rapide

Inconvénients

- Sécurité restreinte par l’emploi des méthodes HTTP

Bilan gRPC

Avantages

- Communication microservices
- Performance critique
- Environnement contrôlé
- Streaming de données

Inconvénients

- Navigateur supporté limité
- Plus complexe
- Cache Manuel

Comment choisir?

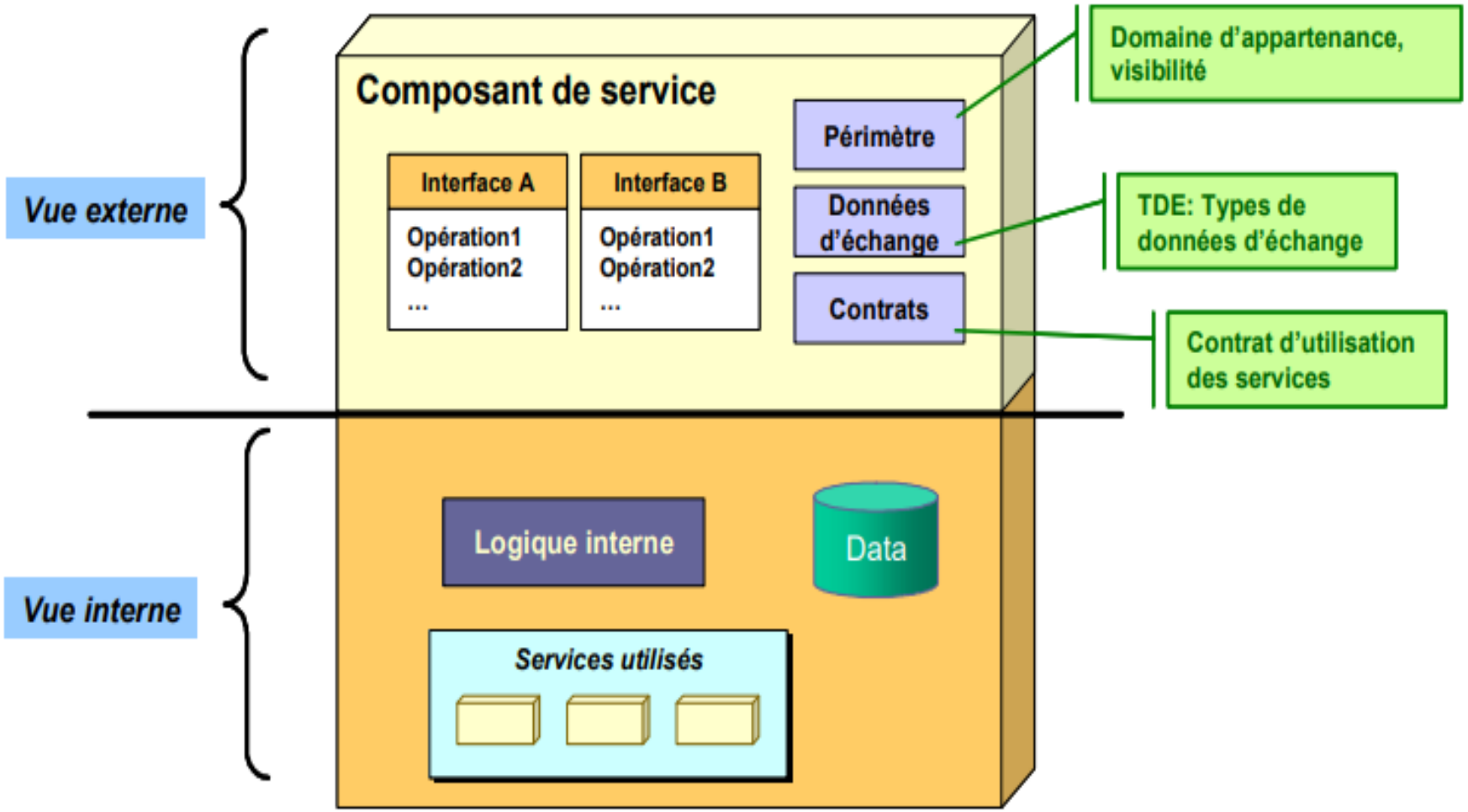
Choisir entre SOAP, REST, gRPC ou tout autre protocole n'est pas seulement une décision technique – c'est une décision commerciale. Les facteurs à considérer incluent :

- Infrastructure existante et compétences au sein de l'organisation
- Exigences d'interopérabilité avec les partenaires ou les clients
- Besoins de conformité réglementaire
- Exigences de performance
- Plans de maintenance et de scalabilité à long terme

Dans certains cas, les normes robustes et la compatibilité ascendante de SOAP peuvent l'emporter sur la simplicité et les avantages de performance des nouveaux protocoles.

Chapitre 5 : Orchestration & Chorégraphie

Composant de service

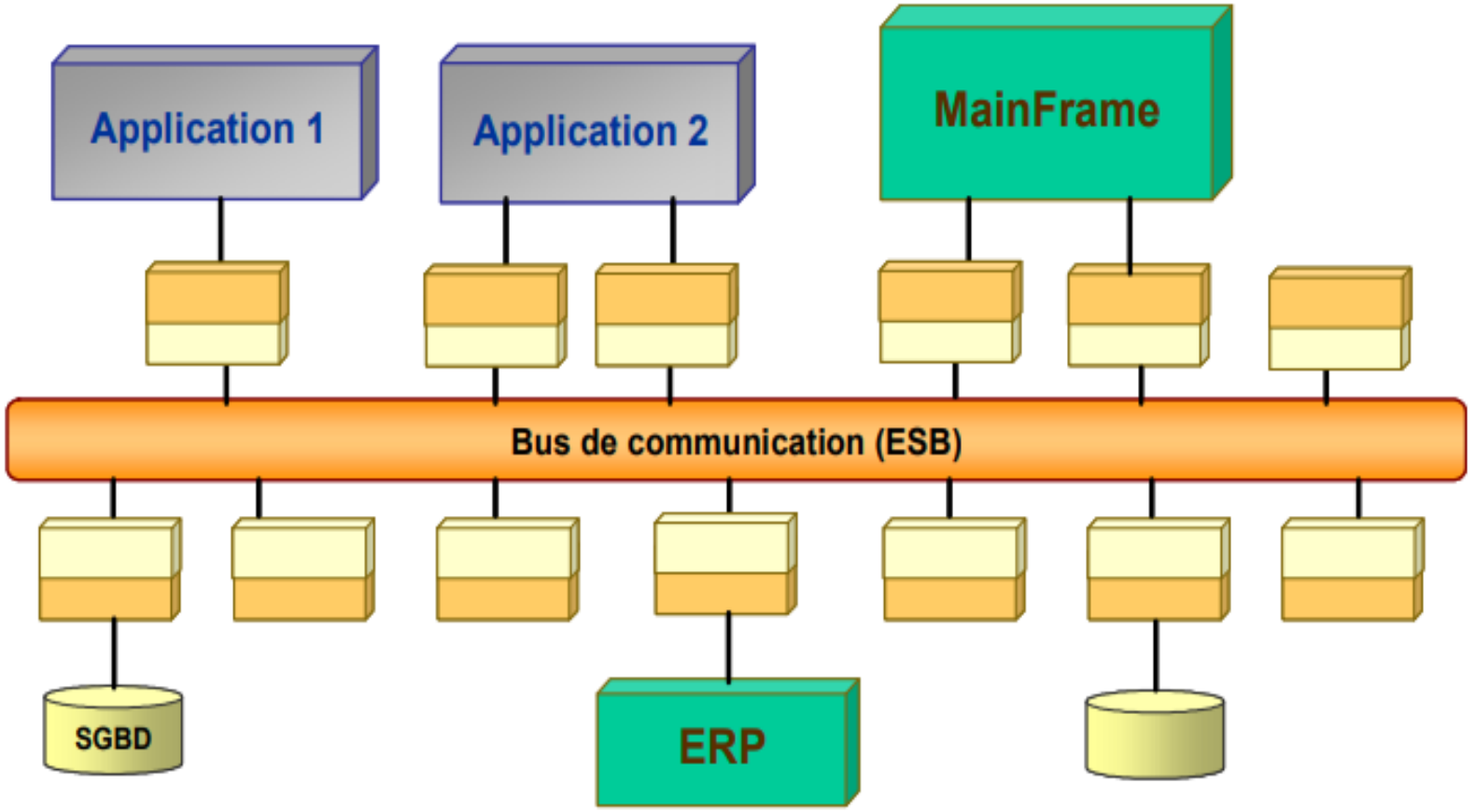


Composant de service

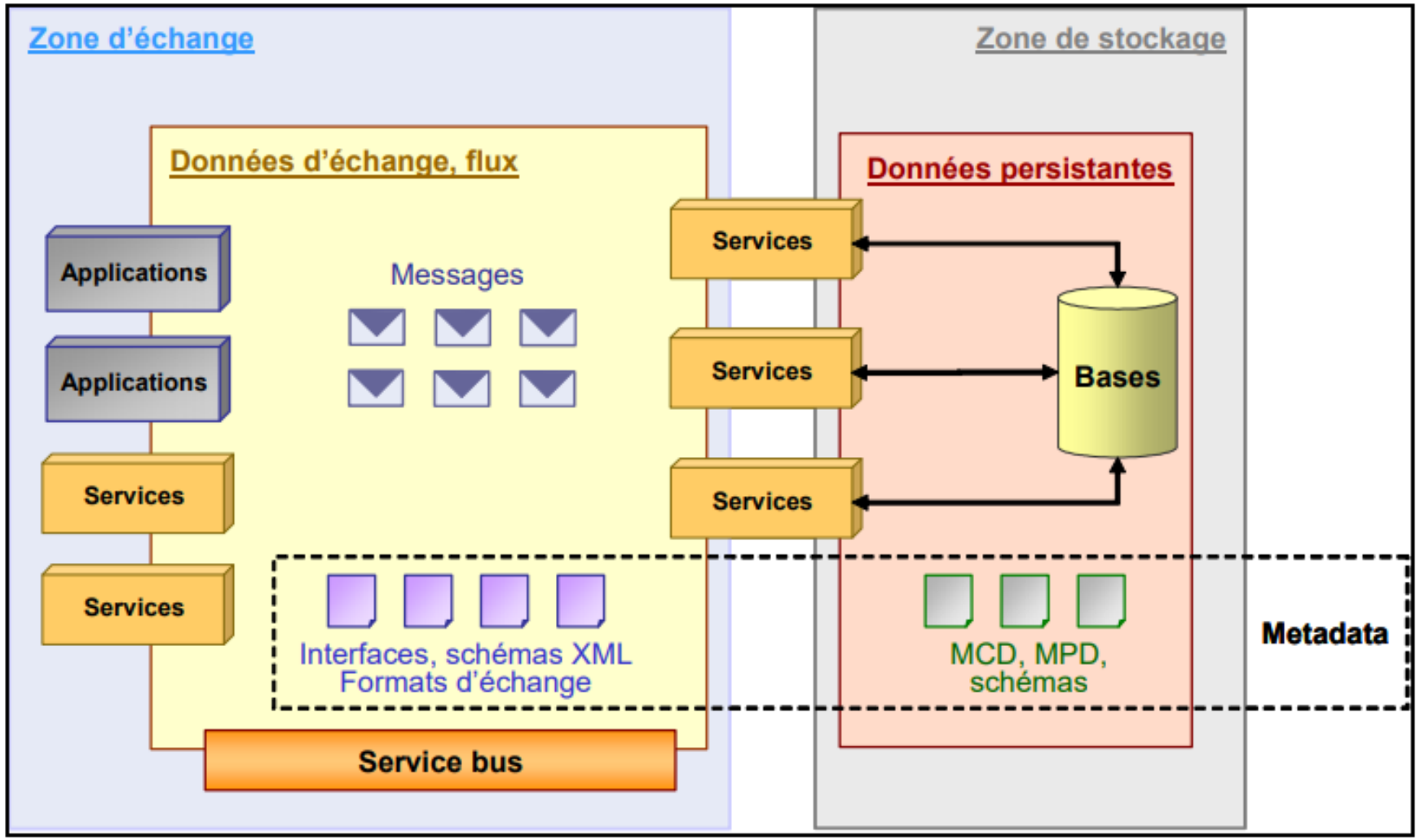
Brique de base de l'architecture, composé d'une vue externe et interne

- La vue externe ou spécification :
  - Expose la facette service proprement dite
  - Constituée :
    - d'un ensemble d'opérations de service regroupées en interfaces
    - appareillage pour les utiliser (types de données échangées, contrat de service, propriétés…)
  - Décrite par un fichier WSDL ou équivalent
- La vue interne :
  - Décrit le contenu du composant
  - Masquée aux consommateurs du composant
  - Contient des informations relatives à la logique interne (détail de traitement ou bases de données) + références vers les services utilisés par le composant

Bus d’entreprise



Données


Zone d'échange
Zone de stockage
Données d'échange, flux
Données persistantes
Applications
Applications
Services
Services
Messages
Services
Services
Services
Bases
Interfaces, schémas XML
Formats d'échange
MCD, MPD,
schémas
Metadata
Service bus

Données

- Données d’échange
  - Informations véhiculées entre les participants à travers l’invocation des opérations de service
  - TDE : Types de donnée d’échange : établissent la sémantique, structure et format de ces données, définis à l’aide de schémas XML ou classes UML
- Données persistantes
  - Informations contenues et gérées dans les bases de données
  - Structurées de façon habituelle (SGBD relationnel, par exemple)

Du service au processus

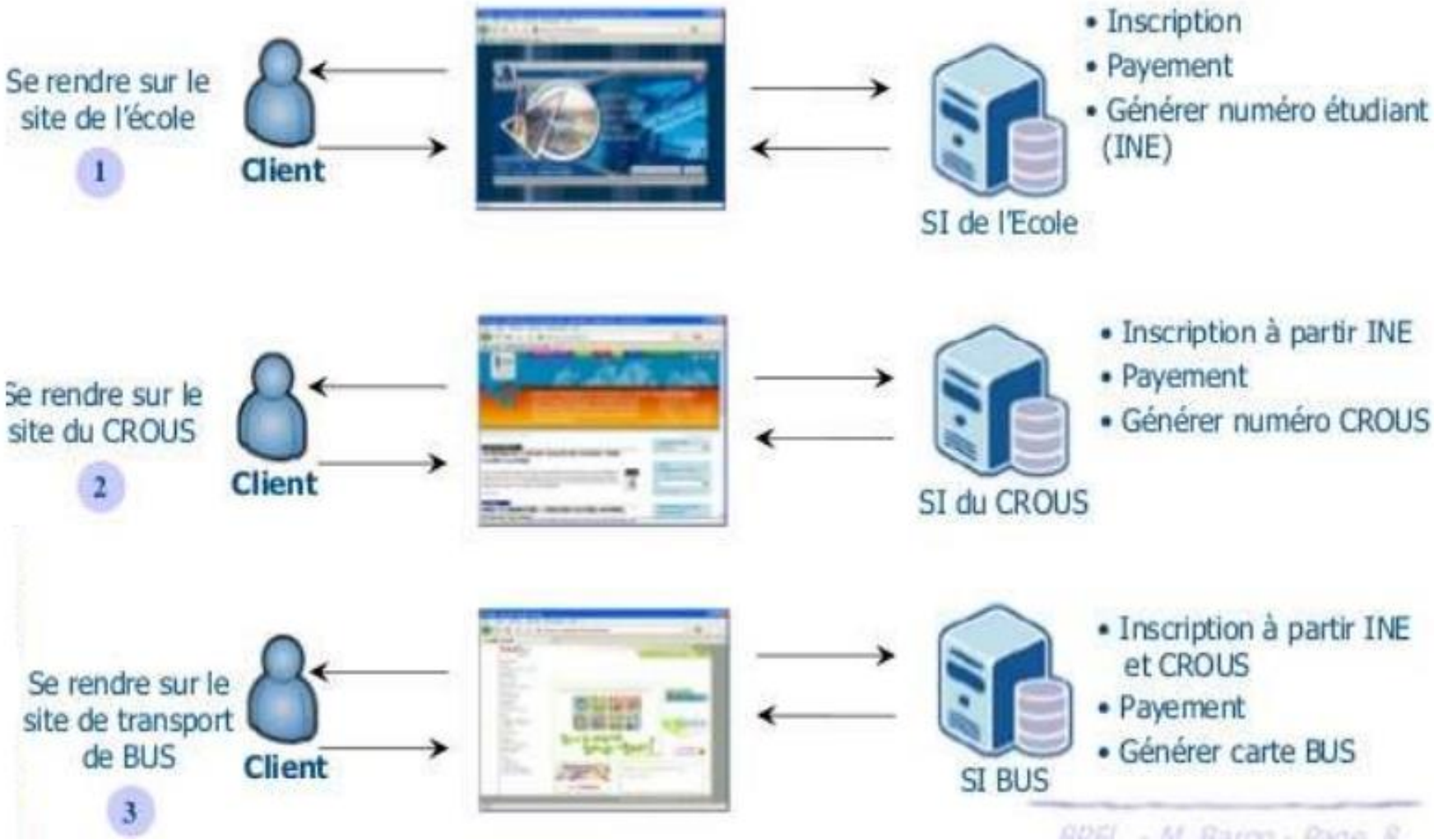
- Le processus (processus métier) est un service plus complexe.
- Le processus est un service qui est le résultat de la composition d’autres services moins complexes.
- Parmi les huit caractéristiques déjà-vus concernant les services, on a dit qu'un service doit être composable, c-à-d qu'il peut être une pièce parmi un ensemble de pièces formant un nouveau service, ce dernier peut lui-même être regroupé avec d'autres, et ainsi de suite.
- Un processus métier peut être vu comme étant une collaboration entre un ensemble de services, dans laquelle (collaboration) chaque service doit intervenir pour accomplir le travail des autres.

Du service au processus

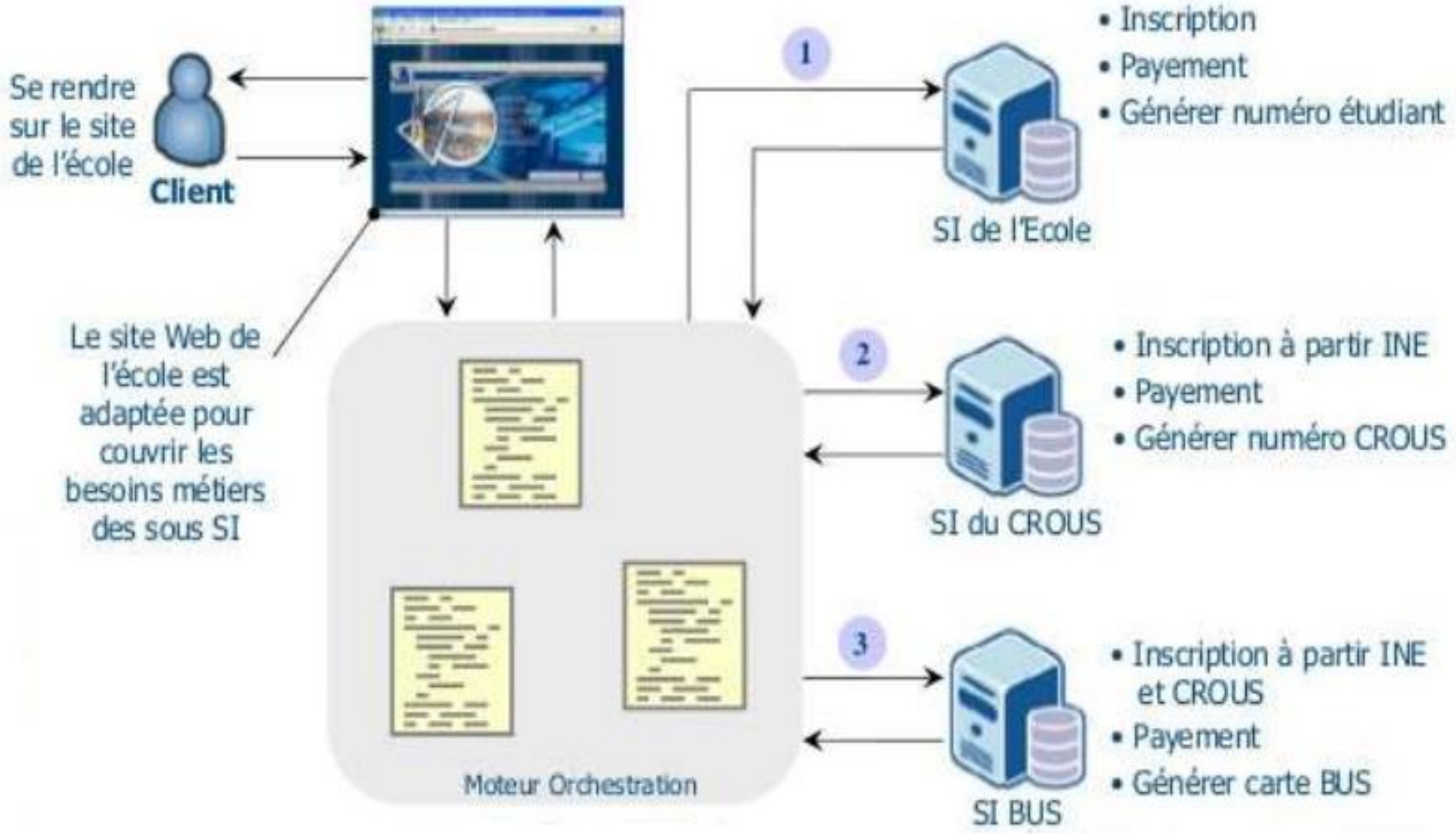
Exemples :

- Inscription d'un étudiant à une école
  - Inscription : nom, adresse…
  - Paiement des frais d'inscription (services d'une banque)
  - Attribution d'un numéro d'étudiant
- Chercher un logement dans un foyer d'étudiants (CROUS)
  - Inscription à partir du numéro d'étudiant
  - Choisir le type de logement
  - Paiement des frais d'hébergement
  - Attribution d'un numéro au foyer
- S'abonner au transport
  - Sélectionner la ligne entre l'école et le logement
  - Choix de la réduction proposée (num d'étudiant)
  - Inscription et Paiement

Du service au processus



Du service au processus



Du service au processus

- Implémentation d'une application (offerte comme service) dont la logique implique l'invocation d'opérations offertes par d'autres services.
  - Le nouveau service est appelé service composite
  - Les services invoqués sont des composants de service
- Du point de vue du client, un service composite et un service basique (implémenté par un langage de programmation traditionnel) sont in-distinguables.
- Notion de Service Mashup :
  - Forme plus légère de composition
  - Pour les applications web

Du service au processus

Utilité:

- Capacité d'invoquer les services d'une manière asynchrone
- Gestion des exceptions et de l'intégrité transactionnelle
  - Les études montrent que presque 80% du temps de construction des processus métier est passé dans la gestion des exceptions
- Fourniture d'un framework dynamique, flexible et adaptable pour une séparation claire entre la logique métier et les services utilisés
- Dépendamment des besoins, la composition des services peut se faire en utilisant deux approches:
  - Orchestration
  - Chorégraphie

Orchestration des services

- Un processus central prend le contrôle sur les web services impliqués et coordonne l'exécution des différentes opérations sur ces web services;
- Les services invoqués ne savent pas qu'ils sont invoqués par un service métier complexe, seul le coordinateur le sait ;
- l'orchestration est centralisée avec des définitions explicites des opérations et l'ordre d'invocation des web services;
- Utilisée par les processus métier privés.

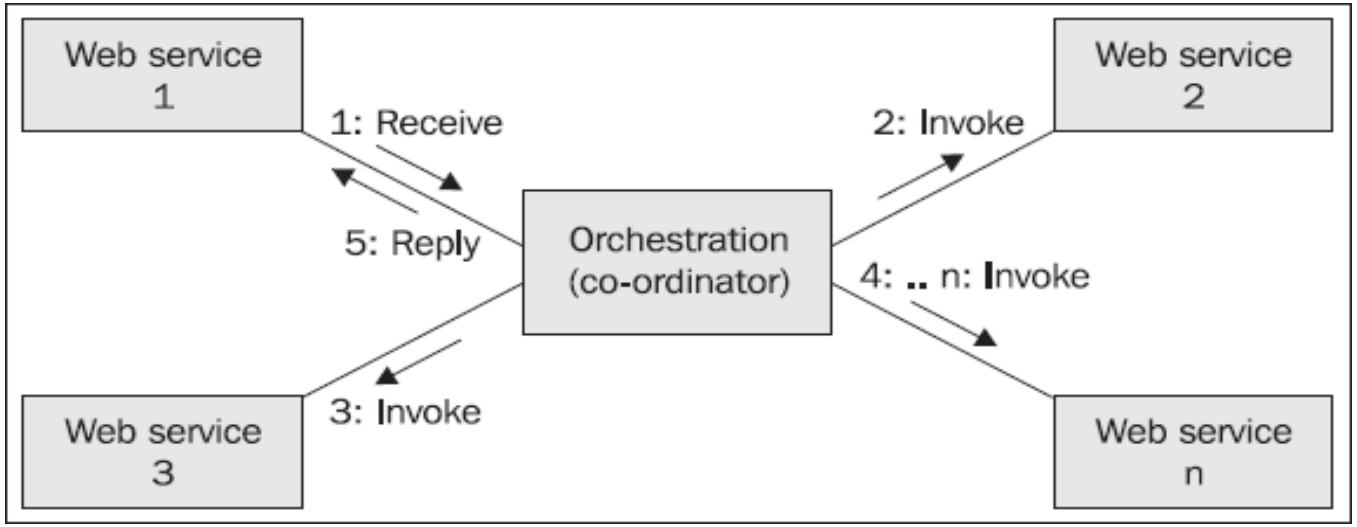
Orchestration des services

- L'orchestration décrit l’interaction entre services, en incluant la logique affaire et l’ordre d’exécution des interactions. Ces interactions peuvent traverser des applications ou/et des organisations, et résulte dans un modèle de processus à multi étapes transactionnel et de longue durée.
- Elle consiste à avoir un service «chef d’orchestre» qui joue le rôle d’intermédiaire entre les services en les appelant suivant un enchaînement prédéfini (script d’orchestration). Ce service est souvent appelé "moteur d'orchestration".
- L’orchestration peut être vue comme une composition ascendante dans laquelle les services web appelés existent au préalable et n’ont pas besoin de savoir qu’ils font partie d’un processus plus gros. C'est uniquement le moteur d'orchestration connaît la logique de composition.

Orchestration des services

- Pour des compositions de services simples, l'orchestration est faite dans le code (Java, C#...) résidant dans le composite
- Pour des orchestrations complexes, un outil est utilisé pour :
  - Créer un modèle visuel d'une séquence
  - Générer le code qui exécute cette séquence dans un environnement d'exécution dédié

⇨Approche BPM (Business Process Model)



Orchestration des services

- BPMN (Business Process Modeling Notation)
  - Succède à BPML (Business Process Modeling Langage)
  - Définit une représentation visuelle de la séquence
- BPEL (Business Process Execution Language) ou BPEL4WS (BPEL for Web Services)
  - Code qui exécute la séquence
  - Exprimé en XML
  - Utilise WSDL à deux niveaux :
    - Interagir avec les ressources requises par le processus
    - Décrire le processus BPEL lui-même
  - Définit deux types d'activités:
    - Activités de base : interagissant avec les services externes (invoke, receive, reply)
    - Activités structurées : contrôle de flux du processus interne (flux séquentiel, condition, boucle…)

Limites de l’orchestration

- Approche centralisée autour du moteur de composition
  - En cas de panne, plus de composition
- Schéma de composition statique
  - En cas de changement dans les besoins, le schéma devient invalide

La chorégraphie

Au lieu d'un coordinateur central, chaque web service impliqué dans la chorégraphie sait exactement quand exécuter ses opérations et avec qui interagir; La chorégraphie est un effort de collaboration focalisé sur l'échange de messages dans des processus métiers publiques; Supporté par le standrd WS-CDL (Web Service Choregraphy Definition Language);

Tous les participants à la chorégraphie doivent savoir :

- leurs rôles dans le processus métier
- les opérations à exécuter
- les messages à échanger
- le temps d'échange de ces messages

La chorégraphie

- WS-CDL (Web Services Choregraphy Description Language) : Succède à WSCI (Web Services Choregraphy Interface)
  - Langage de description en XML
  - Décrit les messages qui sont impliqués dans l'échange collaboratif entre services
  - Ne définit pas de processus métier exécutable (contrairement à BPEL)
  - Un seul document WS-CDL spécifie la contribution d'un partenaire à l'échange de messages.

La chorégraphie

- La chorégraphie trace la séquence des messages qui peuvent impliquer de multiples parties et de multiples sources, en incluant les clients, fournisseurs et partenaires. Elle est typiquement associée à l'échange de messages publics qui se produisent entre de multiples services web plutôt qu'à un processus métier spécifique qui est exécuté par une seule partie.
- La collaboration de services dans une chorégraphie se passe d'une manière décentralisée sans utiliser un procédé principal, c'est-à-dire qu'elle dispense du rôle de chef d'orchestre, mais plutôt plusieurs procédés distribués (services participants à la collaboration) dont chacun d'eux doit être au courant de la logique du (ou des) processus auquel il appartient. Ainsi la logique de contrôle est supervisée par chacun des services intervenant dans la composition et l'exécution du processus est alors distribuée.

La chorégraphie

- Pour concevoir une chorégraphie, les interactions entre les différents services doivent être décrites. Ainsi la description de chaque service intervenant dans la chorégraphie inclut la description de sa participation dans le processus.
- La chorégraphie est aussi appelée composition dynamique, du fait que l'exécution n'est pas régie de manière statique comme dans une l'orchestration. Dans une chorégraphie, à chaque pas de l'exécution (i.e. à chaque étape de la composition), un service web choisit le service web qui lui succède et implémente ainsi une partie de la chorégraphie. La composition de type chorégraphie n'est pas connue, ni décrite à l'avance.

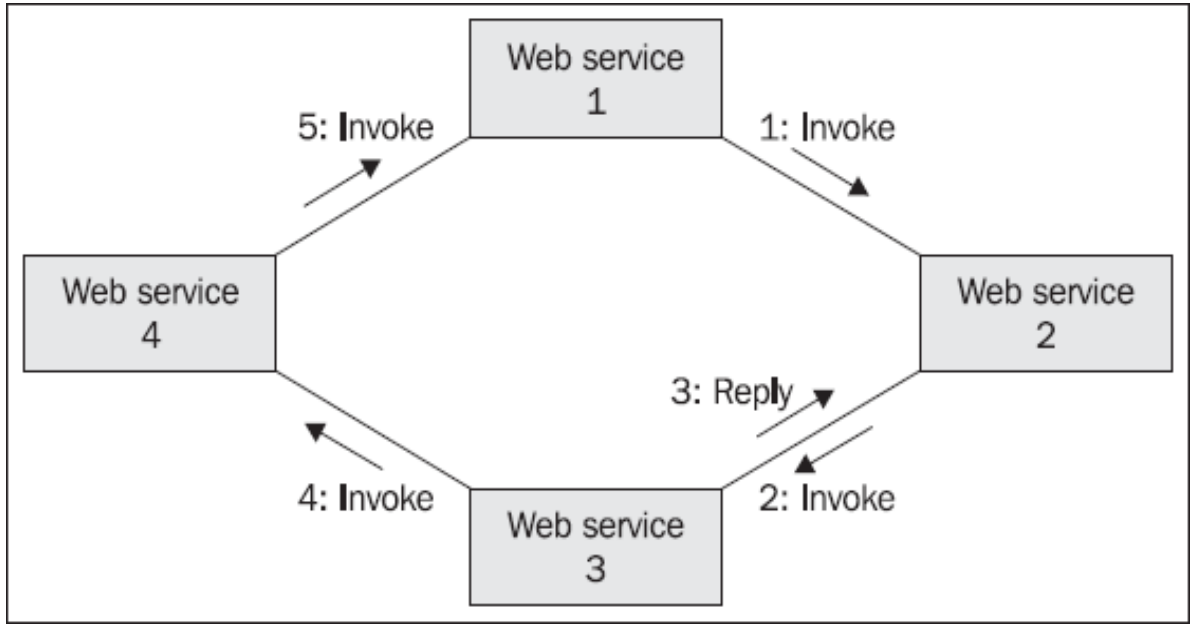
La chorégraphie

WS-CDL (Web Services Choregraphy Description Language) :Succède à WSCI (Web Services Choregraphy Interface)

- Langage de description en XML
- Décrit les messages qui sont impliqués dans l'échange collaboratif entre services
- Ne définit pas de processus métier exécutable (contrairement à BPEL)
- Un seul document WS-CDL spécifie la contribution d'un partenaire à l'échange de messages.

La chorégraphie

- Contient un ensemble d'interactions, représentant les échanges de messages entre parties
- Décrit l'ordre des messages dans lequel ils doivent être observés : Si cet ordre n'est pas respecté, WS-CDL considère les messages comme hors de la séquence, donc erronés



Limites de chorégraphie

- Collaborations et partenaires statiques
  - Si les besoins ou partenaires changent, les collaborations deviennent impossibles
- Pas de langage pour exprimer les besoins
  - Les travaux existant proposent une chorégraphie statique

Orchestration vs Chorégraphie

